Tribhuvan University

Faculty of Humanities and Social Sciences



A Project Report on

StudySmart: An Offline-First Android Application for Academic Study and Revision

Submitted to:

Department of Computer Application

Xavier International College, Boudha, Kathmandu

In partial fulfillment of the requirements for the Bachelor of Computer Application

Submitted by:

Roshani Paudel — University SN: 6-2-546-35-2021

Rehima Pandey — University SN: 6-2-546-34-2021

BCA 8th Semester

Under the Supervision of

Ankit Nepal

CERTIFICATE PAGE

Tribhuvan University

Xavier International College, Boudha, Kathmandu

This is to certify that the project report entitled “StudySmart: An Offline-First Android Application for Academic Study and Revision”, prepared by Roshani Paudel and Rehima Pandey, has been submitted in partial fulfillment of the requirements for the Bachelor of Computer Application at Tribhuvan University. The project was undertaken under the supervision of Ankit Nepal at Xavier International College.

_____

Coordinator, Department of Computer Application

Date: _____

SUPERVISOR RECOMMENDATION

Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College, Boudha, Kathmandu

This is to certify that the project report entitled "StudySmart: An Offline-First Android Application for Academic Study and Revision", prepared by Roshani Paudel and Rehima Pandey in partial fulfillment of the requirements for the Bachelor of Computer Application, has been carried out under the prescribed academic requirements and is recommended for evaluation.

_____

Ankit Nepal

Supervisor, Bachelor of Computer Application

Xavier International College

INTERNAL AND EXTERNAL EXAMINERS' APPROVAL LETTER

Tribhuvan University

Xavier International College, Boudha, Kathmandu

This is to certify that the project prepared by Roshani Paudel and Rehima Pandey entitled "StudySmart: An Offline-First Android Application for Academic Study and Revision", in partial fulfillment of the requirements for the Bachelor of Computer Application, has been evaluated. In our opinion, it is satisfactory in scope and quality as a project for the required degree.

Ankit Nepal Supervisor Bachelor in Computer Application Xavier International College Boudha, Kathmandu	Rajesh Nepal Coordinator Bachelor in Computer Application Xavier International College Boudha, Kathmandu
Internal Examiner	External Examiner

ACKNOWLEDGEMENT

We thank our supervisor, Ankit Nepal, for his guidance throughout the StudySmart project. We are grateful to Xavier International College, Tribhuvan University, faculty members, friends, and our families for their academic support, feedback, and encouragement.

Roshani Paudel and Rehima Pandey

ABSTRACT

StudySmart is an offline-first Android application designed to support university students through structured academic browsing, question preparation, scheduled review, and written mock tests. Its Core Alpha scope serves one local student and organizes curriculum content into courses, academic periods, subjects, chapters, optional topics, and questions. Preparation sessions contain at most 30 unique questions and use Again, Hard, Good, and Easy ratings to record difficulty and schedule subsequent reviews. Written mock tests preserve fixed question snapshots and evaluate responses through transparent weighted keyword matching rather than semantic or AI grading. The application uses Kotlin, Jetpack Compose, ViewModels, repositories, Room, and DataStore to separate interface, processing, and local storage responsibilities. Recorded verification reported a successful debug build, successful lint checks with warnings, curriculum-integrity checks, and 381 passing JVM test cases with no failures or skips. However, live database integration, instrumentation, and manual system testing were not completed during the final audit, while participant acceptance testing and formal performance measurements were not evidenced. StudySmart therefore provides a focused academic study workflow, with further device-level verification required before production use.

TABLE OF CONTENTS

CERTIFICATE PAGE i

SUPERVISOR RECOMMENDATION ii

INTERNAL AND EXTERNAL EXAMINERS' APPROVAL LETTER iii

ACKNOWLEDGEMENT iv

ABSTRACT v

LIST OF ABBREVIATIONS viii

LIST OF FIGURES ix

LIST OF TABLES xi

Chapter 1: Introduction 1

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope and Limitation 2

1.5 Development Methodology 2

1.6 Report Organization 3

Chapter 2: Background Study and Literature Review 4

2.1 Background Study 4

2.2 Literature Review 4

Chapter 3: System Analysis and Design 6

3.1 System Analysis 6

3.2 System Design 22

3.3 Algorithm Details 25

Chapter 4: Implementation and Testing 29

4.1 Implementation 29

4.2 Testing 31

4.3 Result Analysis 36

Chapter 5: Conclusion and Future Recommendations.....37

5.1 Conclusion 37

5.2 Future Recommendations 37

References.....38

Appendices.....39

Appendix A: Application Screenshots 39

Appendix B: Source Code 46

LIST OF ABBREVIATIONS

Abbreviation	**Meaning**
AGP	Android Gradle Plugin
API	Application Programming Interface
APK	Android Package
BCA	Bachelor of Computer Application
DAO	Data Access Object
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
IDE	Integrated Development Environment
JDK	Java Development Kit
JVM	Java Virtual Machine
KSP	Kotlin Symbol Processing
NFKC	Normalization Form Compatibility Composition
OCR	Optical Character Recognition
SDK	Software Development Kit
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines

LIST OF FIGURES

Figure No.	Title
Figure 3.1	Proposed StudySmart Workflow
Figure 3.2	StudySmart Use-Case Overview
Figure 3.3	Team-Supplied Project Schedule, April–October 2026
Figure 3.4	Curriculum Class Model
Figure 3.5	Question Learning-State Model
Figure 3.6	Course and Academic-Period Selection Sequence
Figure 3.7	Preparation Rating Sequence
Figure 3.8	Mock-Test Submission Sequence
Figure 3.9	Preparation Activity Diagram
Figure 3.10	Due-Review Activity Diagram
Figure 3.11	Mock-Test Activity Diagram
Figure 3.12	Context Diagram / Level 0 DFD
Figure 3.13	Level 1 Data Flow Diagram
Figure 3.14	Layered Application Architecture
Figure 3.15	Supplied Database ERD (Broader Design)
Figure 3.16	Penpot Interface Design Reference
Figure 3.17	Preparation Selection and Queue Flowchart
Figure 3.18	Keyword Evaluation Flowchart
Figure 4.1	Testing and Verification Workflow
Figure A.1	Application Home / Navigation
Figure A.2	Courses Screen
Figure A.3	Academic-Period Screen
Figure A.4	Subjects and Chapters Screen
Figure A.5	Preparation Screens
Figure A.6	Preparation Summary Screens
Figure A.7	Review Screen
Figure A.8	Test Defaults Screen
Figure A.9	Mock-Test Screen

Figure No.	Title
Figure A.10	Test-Result Screen
Figure A.11	Progress Screen (Broader Functionality)
Figure A.12	Important Questions Screen (Broader Functionality)
Figure A.13	Search Screen (Broader Functionality)
Figure A.14	Settings Screen (Broader Functionality)
Figure B.1	Supplied Android Studio Project Structure

LIST OF TABLES

Table No.	Title
Table 2.1	Comparison of Related Study Systems
Table 3.1	Functional Requirements
Table 3.2	UC-01: Select Academic Context
Table 3.3	UC-02: Complete Preparation
Table 3.4	UC-03: Complete Due Review
Table 3.5	UC-04: Create Mock Test
Table 3.6	UC-05: Submit Mock Test
Table 3.7	UC-06: View Test Result
Table 3.8	UC-07: Retest Weak Questions
Table 3.9	Non-Functional Requirements
Table 3.10	Core Alpha Data Entities
Table 3.11	Deterministic Review Intervals
Table 4.1	Implementation Tools and Platforms
Table 4.2	Module Implementation Responsibilities
Table 4.3	Representative Unit-Test Cases and Reported Evidence
Table 4.4	Consolidated System-Test Cases — Manual Verification Required
Table 4.5	Recorded Verification Results

Chapter 1: Introduction

1.1 Introduction

StudySmart addresses examination preparation for university students who otherwise manage questions, revision schedules, and written practice across separate notes and files. The Android application organizes local curriculum content and connects browsing, question preparation, due review, written tests, and result inspection in one offline workflow.

1.2 Problem Statement

A manual study process does not consistently record attempted questions, difficulty ratings, next-review dates, or written-test evidence. Students must choose what to revisit and check answers themselves. StudySmart provides a limited local workflow that records these activities without requiring accounts, a server, or continuous internet access.

1.3 Objectives

General Objective

To develop an offline-first Android application for structured academic browsing, question preparation, scheduled review, and written self-assessment using transparent keyword-based results.

Specific Objectives

• Support database-driven semester/year navigation through courses, academic periods, subjects, chapters, optional topics, and questions.

• Create bounded preparation sessions with answer reveal, four difficulty ratings, immutable review history, and deterministic scheduling.

• Create fixed written mock tests using validated defaults, saved answer drafts, optional timing, and frozen question evidence.

• Present keyword-match scores, review recommendations, weak-question retests, and a bounded readiness estimate.

• Preserve committed learning data, handle empty/error states safely, and verify domain and application behaviour through appropriate tests.

1.4 Scope and Limitation

Scope

Core Alpha targets one local student using an English, portrait-oriented Android phone interface, with minimum API 26 and compile/target API 35. Its academic hierarchy is Course → Academic Period → Subject → Chapter → optional Topic → Question. Questions belong to a chapter and may optionally belong to one of that chapter's topics. Course structures and period counts come from stored data, not hard-coded course names.

The delivered learning scope includes chapter preparation, due review, written mock tests, test defaults, draft persistence, keyword results, weak-question retests, and a compact readiness estimate. A preparation session and a mock test contain at most 30 unique questions.

Limitations

Core Alpha excludes authentication, multiple accounts, backend APIs, synchronization, cloud backup, teacher/admin access, notifications, additional question types, and semantic grading. Full search, bookmarks, progress dashboards, and settings remain outside the reported Core Alpha scope. Data collection used OCR separately; automatic PDF/OCR ingestion is not an application feature.

Keyword matches do not prove semantic correctness or replace teacher evaluation. Committed reviews and submitted results are intended to persist, but unfinished active work is removed by startup cleanup after process death. The dataset and single-user scope limit generalization; device-level acceptance and formal benchmarking remain outstanding.

1.5 Development Methodology

The team used iterative, incremental development. Each iteration defined a small behaviour, designed its data and interface, implemented it, ran targeted checks, corrected defects, and integrated the change. Work progressed from local storage and navigation to

preparation, review, fixed tests, evaluation, and result presentation. This dependency order kept the academic deliverable focused on a working local learning loop.

1.6 Report Organization

Chapter 1 introduces the project and its boundaries. Chapter 2 relates learning research and similar systems to StudySmart. Chapter 3 explains system requirements, feasibility, models, design, and algorithms. Chapter 4 describes implementation and testing results. Chapter 5 presents the conclusion and future recommendations. References and appendices provide cited sources, screenshots, and source-code information.

Chapter 2: Background Study and Literature Review

2.1 Background Study

StudySmart applies retrieval practice by presenting a question before revealing its answer. Roediger and Karpicke found that memory tests can improve later retention, supporting this question-first workflow [1]. After reveal, the student records a difficulty rating rather than receiving an unsupported claim that recall was objectively assessed.

Cepeda et al. found that distributing practice over time influences retention [2]. StudySmart uses this principle through deterministic review intervals: Again = 10 minutes, Hard = 1 day, Good = 3 days, and Easy = 7 days. These are project-defined rules, not scientifically optimized intervals.

Crompton and Burke's higher-education review supports mobile devices as a setting for learning activities [3]. StudySmart contextualizes mobile learning through locally stored academic content and history, allowing preparation and testing without continuous network access. Written self-assessment supplements flashcards; keyword matching provides an explainable heuristic rather than semantic grading.

2.2 Literature Review

Anki supports customizable flashcards, local study, and scheduled review [4]. Quizlet provides flashcard sets and practice-test activities [5]. These systems are useful comparison points, but StudySmart does not claim to outperform them. Its contribution is the selected combination of academic hierarchy, deterministic due review, fixed written-test snapshots, and keyword evidence in a small local Android application.

Table 2.1: Comparison of Related Study Systems

Criterion	**Anki [4]**	**Quizlet [5]**	**StudySmart**
Organization	User-created decks	Study sets	Course/period hierarchy
Preparation	Flashcard review	Flashcards/study modes	Reveal and rate questions

Criterion	Anki [4]	Quizlet [5]	StudySmart
Review	Scheduled flashcards	Multiple study activities	Fixed rating intervals
Written practice	Not primary workflow	Practice-test activities	Chapter written tests
Result evidence	Recall ratings	Activity-dependent	Frozen keyword matches
Local operation	Local study; optional sync	Offline equivalence not assumed	Local Room/ DataStore

The research supports retrieval, distributed practice, and mobile study as motivations. It does not validate StudySmart's particular interval rules, weighted scores, or readiness formula. Those rules are treated as transparent, testable project decisions.

Chapter 3: System Analysis and Design

3.1 System Analysis

Existing and Proposed Workflow

The existing workflow is a generalized manual process: locate materials, select questions, study or write answers, decide what to revisit, and check responses. No interviews, questionnaires, or observation studies were conducted. Requirements were established through discussions between the two team members. Curriculum text was collected from teacher-supplied and online PDFs using OCR; extraction accuracy and reuse permissions were not established.

The proposed workflow selects a course and academic period, browses subjects/chapters, starts preparation or a written test, commits ratings or submitted answers, and returns to due review or results.

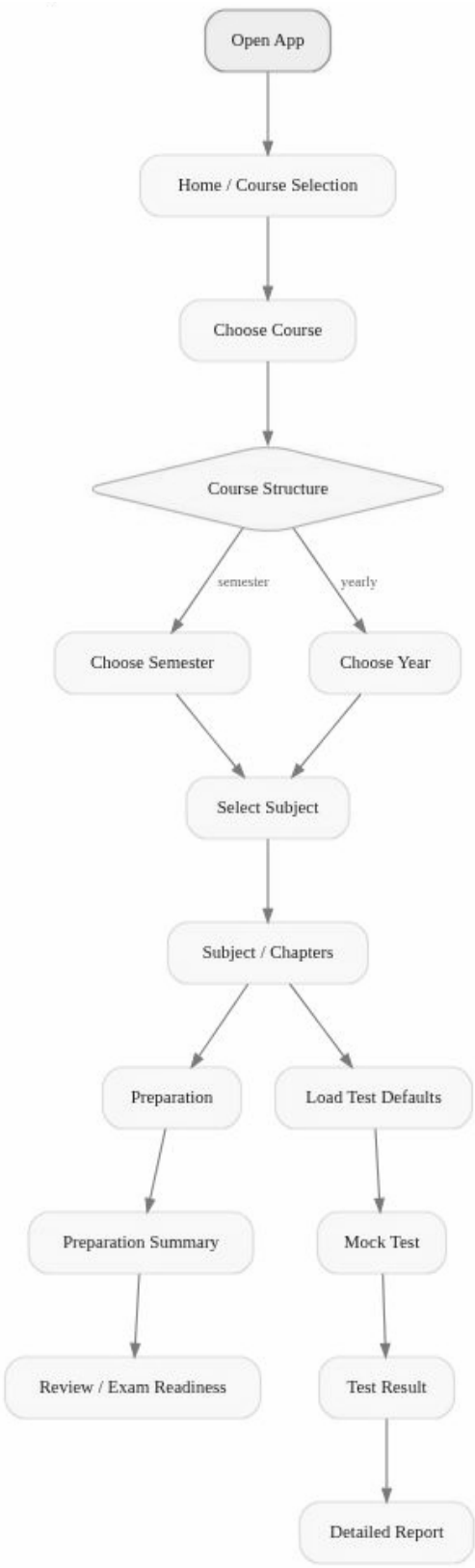


Figure 3.1: Proposed StudySmart Workflow

Requirement Analysis

Functional Requirements

Table 3.1: Functional Requirements

ID	Required Behaviour
FR-01	Open as a local Android application without authentication.
FR-02	List active courses from local storage.
FR-03	Support semester-based and year-based periods.
FR-04	List subjects and chapters within the selected hierarchy.
FR-05	Persist a preparation session before opening its active route.
FR-06	Limit preparation to 30 unique questions.
FR-07	Require answer reveal before accepting a rating.
FR-08	Store each committed rating as an immutable event.
FR-09	Apply Again and first-Hard requeue rules.
FR-10	Calculate and store the next-review instant.
FR-11	Create preparation sessions from eligible due questions.
FR-12	Validate and persist test defaults.
FR-13	Create a fixed attempt before opening a mock test.
FR-14	Support written answers, navigation, flags, and optional timing.
FR-15	Persist drafts before navigation and submission.
FR-16	Evaluate answers using frozen keyword evidence.
FR-17	Display score, keyword match, time, and review evidence.
FR-18	Create an independent weak-question retest.
FR-19	Preserve committed reviews and submitted results after restart.
FR-20	Remove unfinished active work during startup cleanup.
FR-21	Provide safe loading, empty, error, retry, and missing-content states.

The student is the human actor. The database and clock are internal supporting services, not independent user roles. Figure 3.2 retains the supplied broader use-case overview; search, important questions, and full progress shown there remain outside the reported Core Alpha scope.

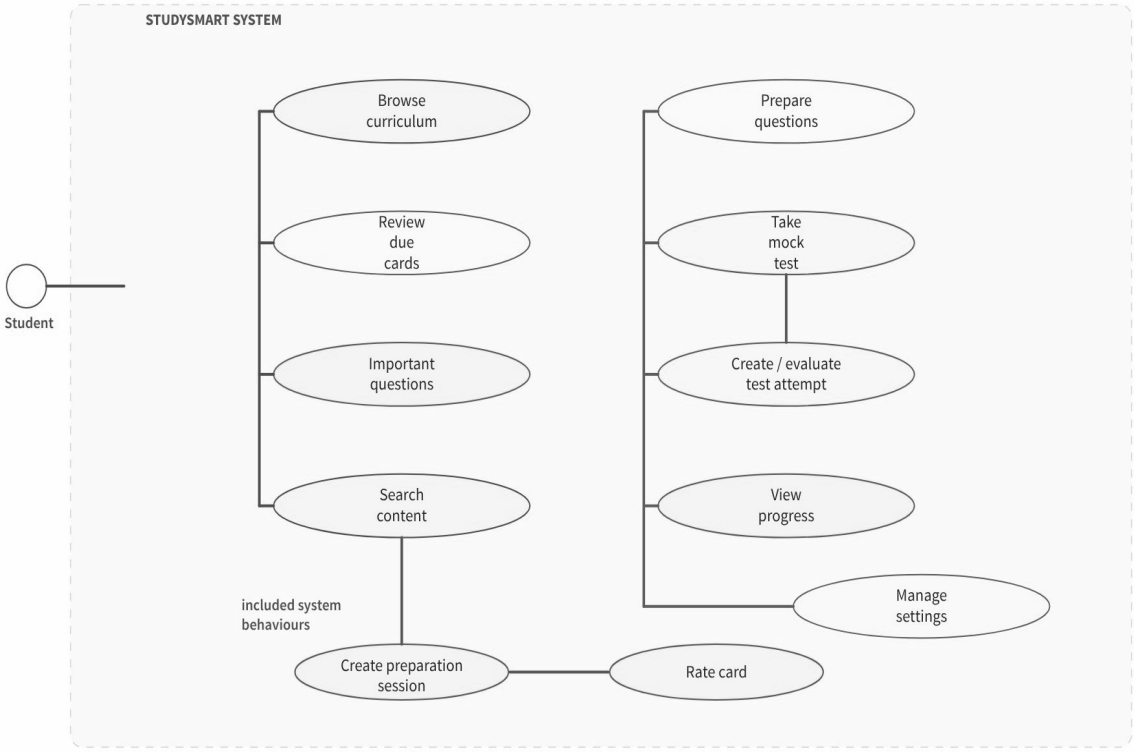


Figure 3.2: StudySmart Use-Case Overview

Use-Case Descriptions

Table 3.2: UC-01: Select Academic Context

Field	Description
Preconditions	Active course data exists.
Trigger	Students open Courses or need to select context.
Main flow	Select course, view its stored periods, select a period, save the valid pair, and return to the originating context.
Alternative flow	If there are no courses or periods, show an empty state. If an ID is stale, show recovery without replacement selection.
Postcondition	Selected courses and periods are stored in the DataStore .

Table 3.3: UC-02: Complete Preparation

Field	Description
Preconditions	A selected chapter has eligible questions.
Trigger	Student selects Prepare.
Main flow	Create a fixed session, show question, reveal answer, accept rating, persist history and schedule, apply queue rule, and continue until queue completion.
Alternative flow	Empty pool creates no session. Save failure blocks advancement and offers Retry. Close asks for Discard or Cancel.
Postcondition	Committed ratings remain and summary is shown.

Table 3.4: UC-03: Complete Due Review

Field	Description
Preconditions	Valid selected course and period.
Trigger	Student opens Review and selects Start Due Review.
Main flow	Query due questions, order them by due time, create a preparation session, and reuse the preparation flow.
Alternative flow	No due questions produces the NoDue state. Missing context produces a context-required state.
Postcondition	Due questions can be rated and rescheduled.

Table 3.5: UC-04: Create Mock Test

Field	Description
Preconditions	A chapter contains eligible written questions.
Trigger	Student selects Test.
Main flow	Load defaults, build eligible pool, select up to the requested count, create frozen attempt snapshots and answer rows, and navigate to the test.
Alternative flow	No eligible questions creates no attempt and shows NoTestQuestions. Fewer available questions creates a smaller valid attempt.

Field	Description
Postcondition	One fixed attempt exists before the test screen opens.

Table 3.6: UC-05: Submit Mock Test

Field	Description
Preconditions	An attempt is in progress.
Trigger	Student confirms Submit, including after an expiry prompt.
Main flow	Flush drafts, show counts, transition to submitting, evaluate frozen inputs, store evidence and totals, and mark submitted.
Alternative flow	Expiry freezes the draft and offers Submit or Discard; it does not automatically submit. Repeated submission is idempotent. Save failure blocks submission.
Postcondition	Completed attempt is read-only and available to the result screen.

Table 3.7: UC-06: View Test Result

Field	Description
Preconditions	Submitted attempt exists.
Trigger	Submission completes or the student opens a result.
Main flow	Display score, keyword match, time, review count, and per-question frozen evidence.
Alternative flow	No weak questions disables Retest Weak with explanatory text. Missing attempt shows recovery rather than a replacement attempt.
Postcondition	Students can return or create a weak-question retest.

Table 3.8: UC-07: Retest Weak Questions

Field	Description
Preconditions	Submitted source attempt contains active review-required questions.
Trigger	Student selects Retest Weak Questions.
Main flow	Select the source attempt's review-required subset, create a new fixed attempt, and open it.
Alternative flow	Empty subset disables the action and displays an explanation.

Field	Description
Postcondition	A new attempt is independent of the source attempt.

All seven use cases have the Student actor. Persistence failures must not silently advance a preparation card or test question. Empty or stale contexts display a recovery state rather than selecting replacement data automatically.

Non-Functional Requirements

Table 3.9: Non-Functional Requirements

Area	Project Requirement
Performance	Run storage off the main UI thread. Startup/interaction budgets are unmeasured targets, not achieved results.
Security/privacy	Store no passwords/tokens; exclude learning data from backup/ transfer; do not log answers, keywords, history, or scores.
Usability	Follow academic navigation; use clear labels and recoverable empty states; confirm destructive discard actions.
Accessibility	Support semantic labels, approximately 48 dp targets, font scaling, wrapping, and scrolling; WCAG 2.2 AA is a release target.
Reliability	Use atomic creation/rating/submission/cleanup; handle duplicates idempotently; keep submitted evidence immutable.
Maintainability	Separate UI, ViewModel, domain, repository, and persistence; inject clock/random sources and use stable route IDs.
Scale boundary	Support one local student; concurrent-user scalability and backend synchronization are outside scope.

Feasibility Analysis

Technical Feasibility

Kotlin, Compose, Room, and DataStore support the required local UI, storage, and deterministic business rules without a server or scoring API. The minimum client platform is Android API 26. Excluding accounts and synchronization reduces integration dependencies.

Operational Feasibility

A single student can browse a selected academic context and study without registration or network setup. Familiar navigation and explicit reveal/rating actions support the intended workflow. Participant UAT was not evidenced, so operational acceptance is not claimed.

Economic Feasibility

The team reported no additional project expenditure and used available development resources. The principal contribution was the two students' time; no monetary valuation was recorded. Core Alpha requires neither a paid backend nor an external scoring API.

Schedule Feasibility

The team-supplied April–October 2026 schedule overlaps analysis, design, database preparation, development, integration, testing, and reporting. The limited Core Alpha scope supports this incremental schedule. A scheduled testing period does not establish that every verification level was completed.

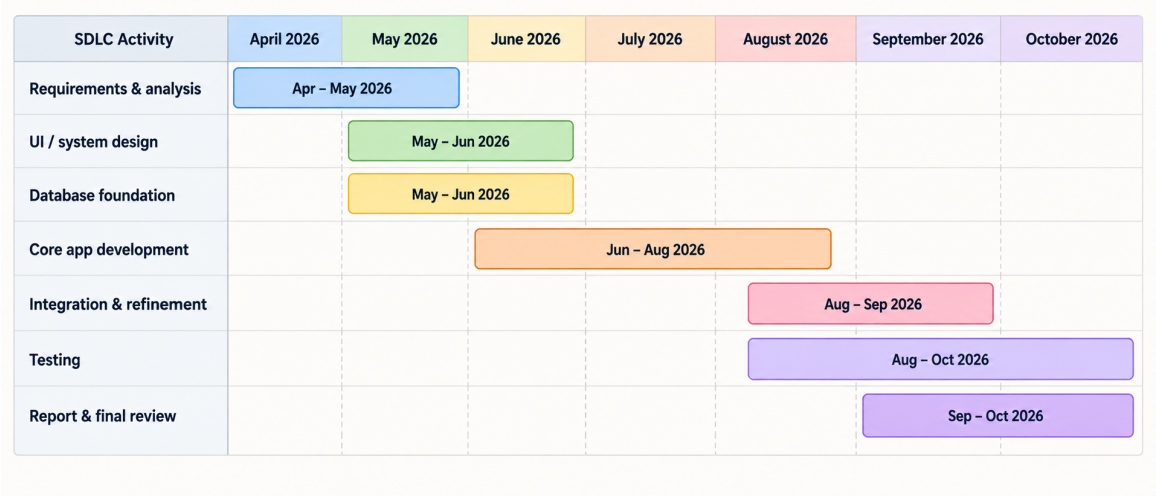


Figure 3.3: Team-Supplied Project Schedule, April–October 2026

Object Modelling

The curriculum class model relates Course, AcademicPeriod, Subject, Chapter, optional Topic, Question, and QuestionKeyword. Stored IDs link each child to its parent. Questions may belong directly to a chapter, so artificial topic records are unnecessary. Figure 3.4 is the existing class model, originally captioned as the academic hierarchy. No separate object-instance diagram was supplied.

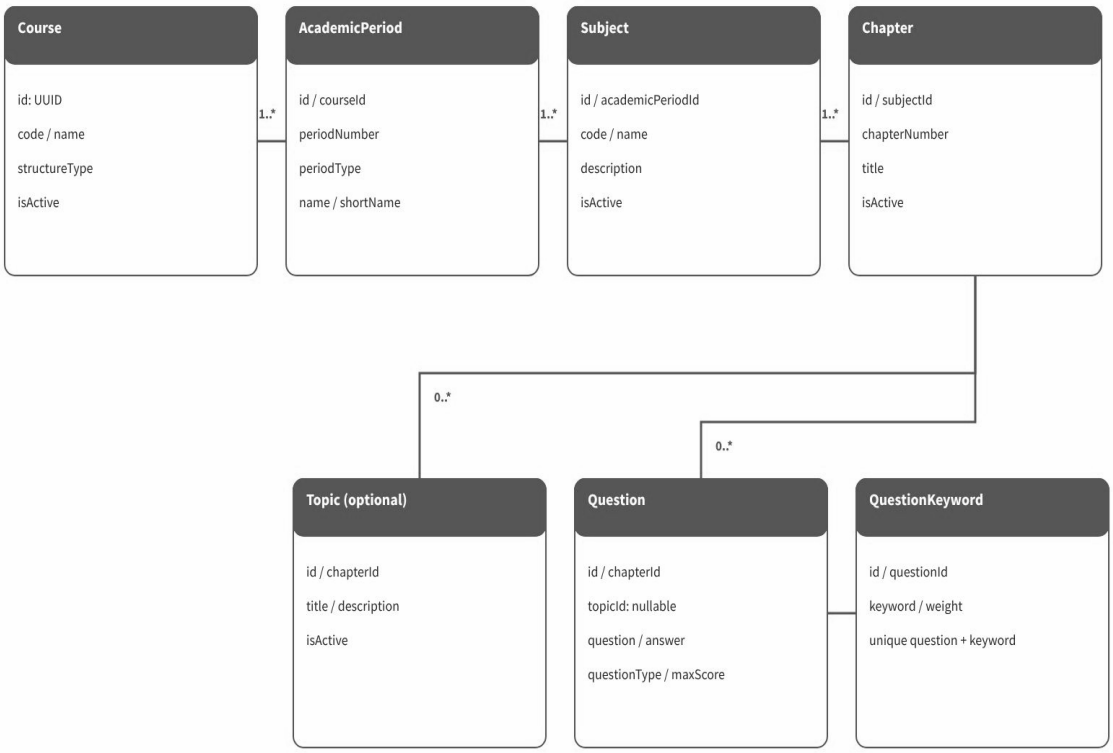


Figure 3.4: Curriculum Class Model

Dynamic Modelling: State and Sequence Diagrams

The supplied learning-state model depicts transitions between study states and due review. It is a conceptual design artifact, not evidence that every depicted state transition was verified on a device. Sequence diagrams show academic-context selection, preparation rating, and mock-test submission through UI, domain, repository, and storage boundaries.

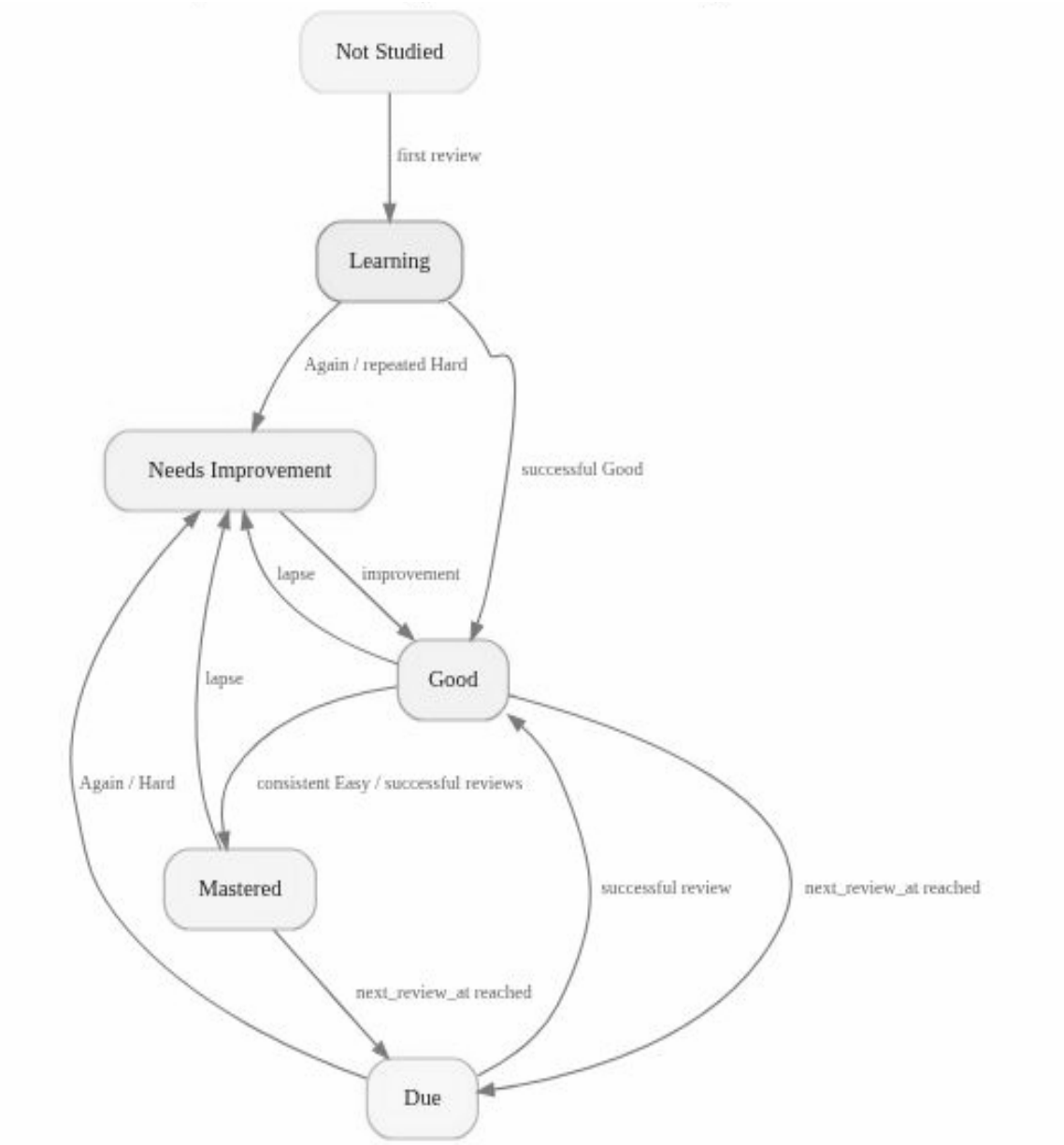


Figure 3.5: Question Learning-State Model

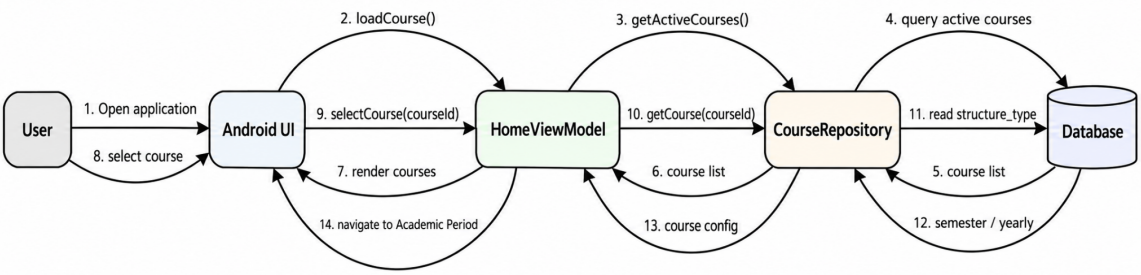


Figure 3.6: Course and Academic-Period Selection Sequence

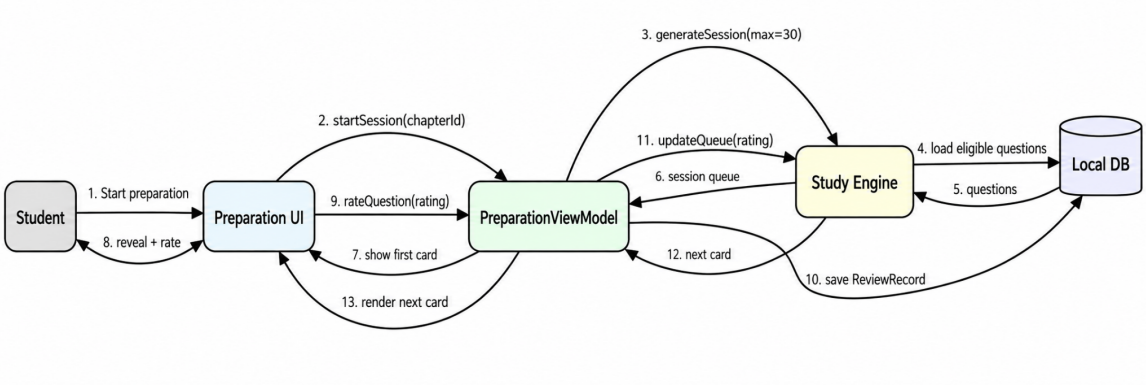


Figure 3.7: Preparation Rating Sequence

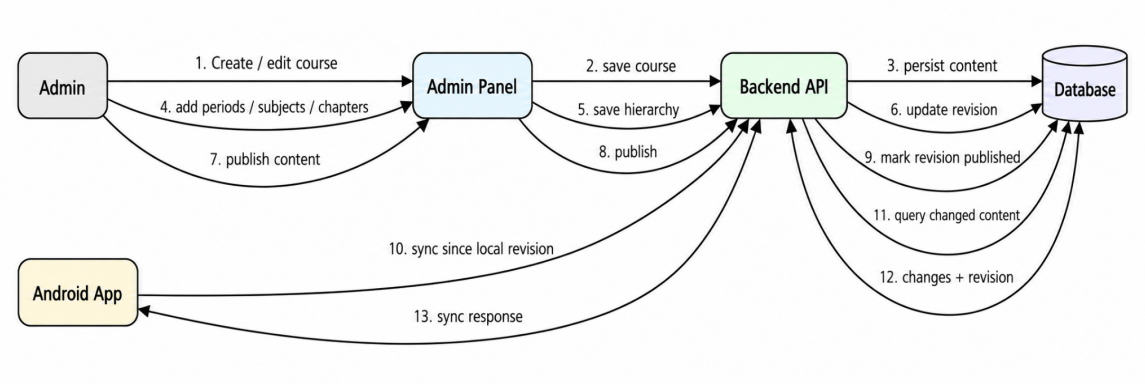


Figure 3.8: Mock-Test Submission Sequence

Process Modelling: Activity Diagrams

Activity diagrams describe preparation, due review, and written-test flows, including no-content branches and completion. The retained context and Level 1 DFDs describe exchanges with local curriculum, preferences, learning history, and result data.

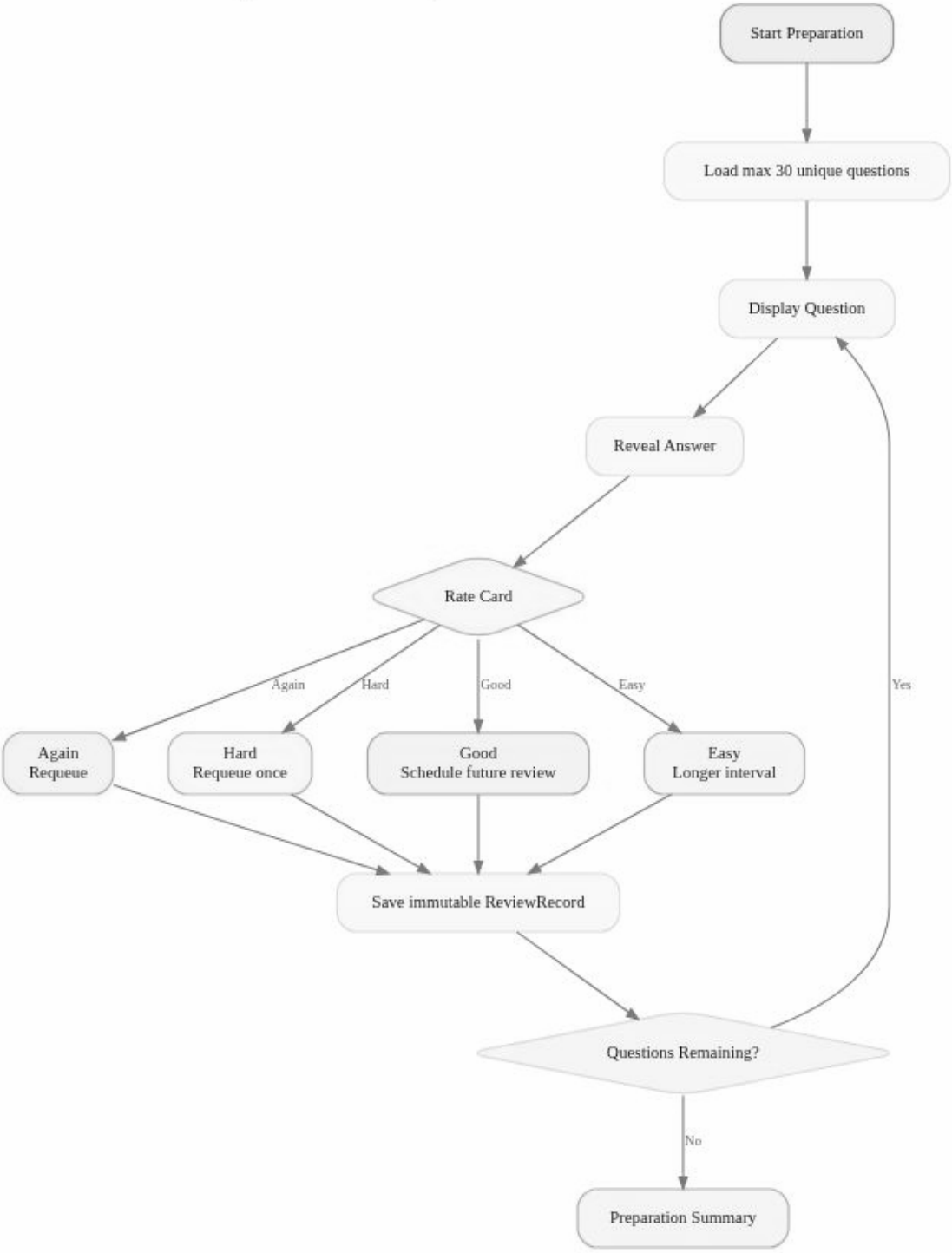


Figure 3.9: Preparation Activity Diagram

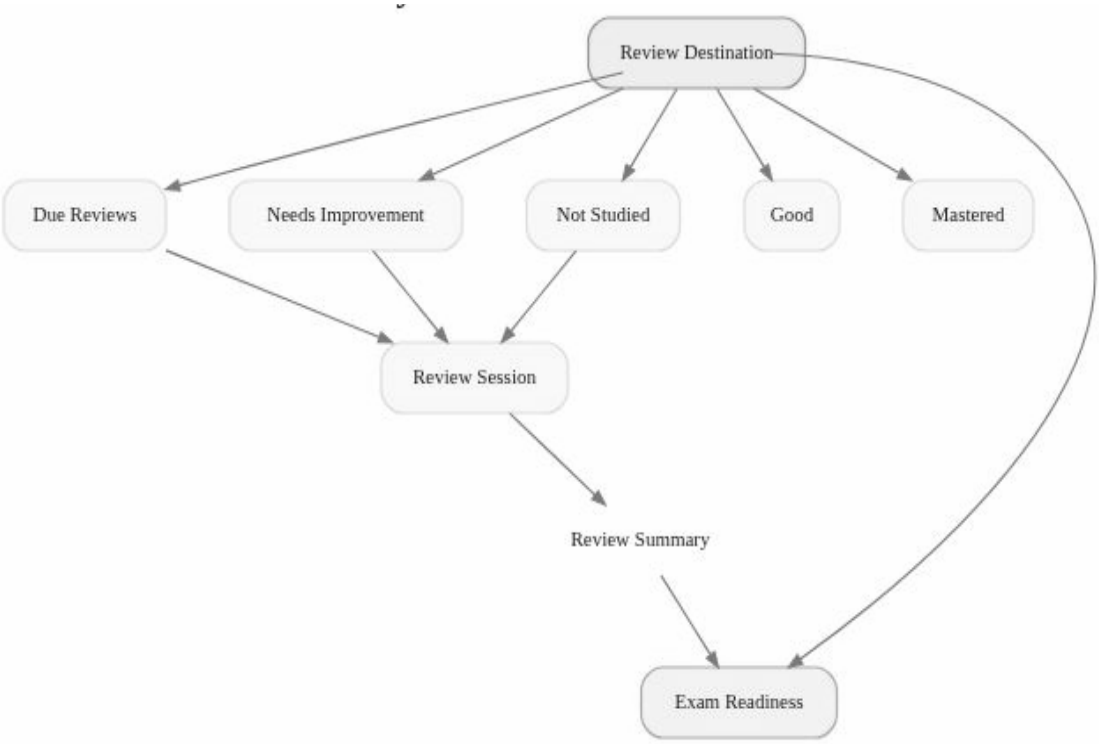


Figure 3.10: Due-Review Activity Diagram

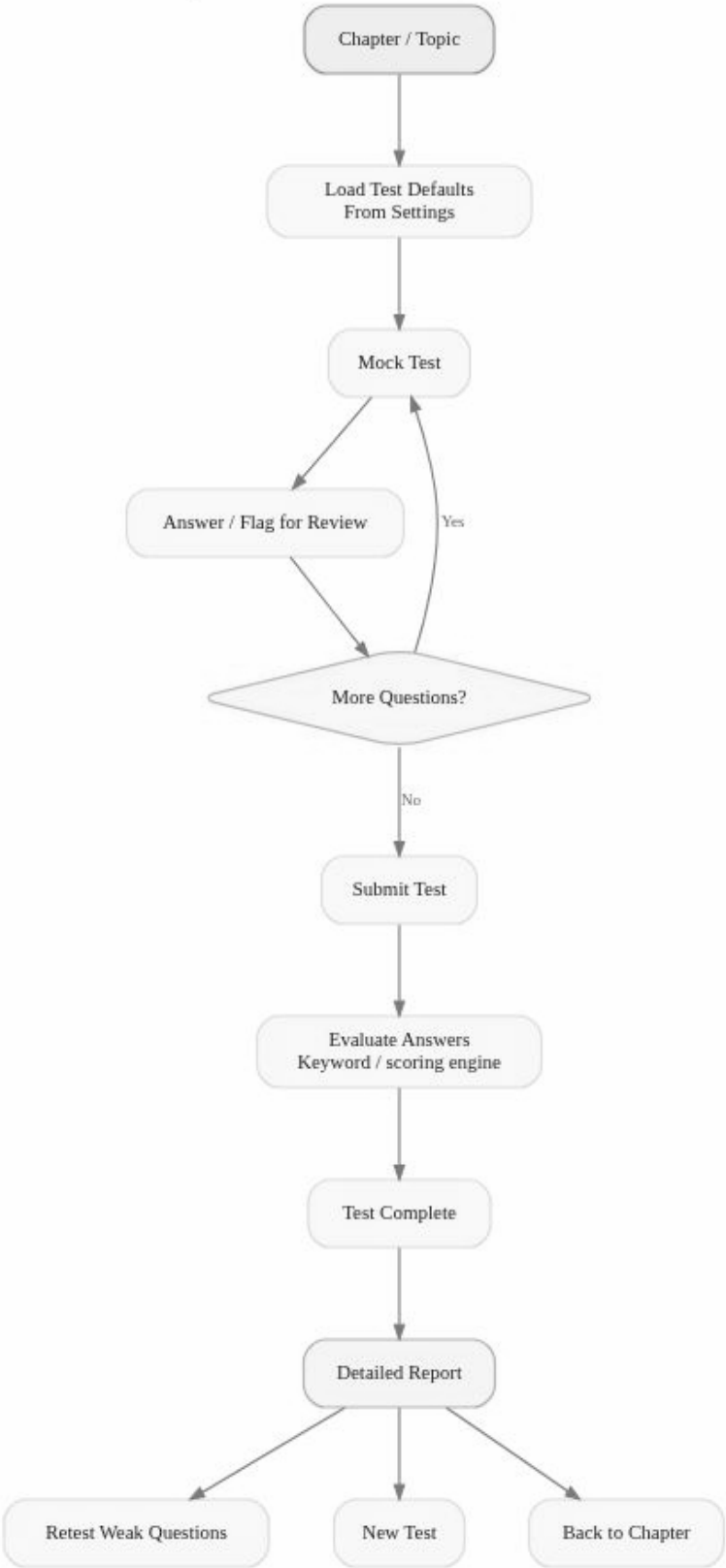


Figure 3.11: Mock-Test Activity Diagram

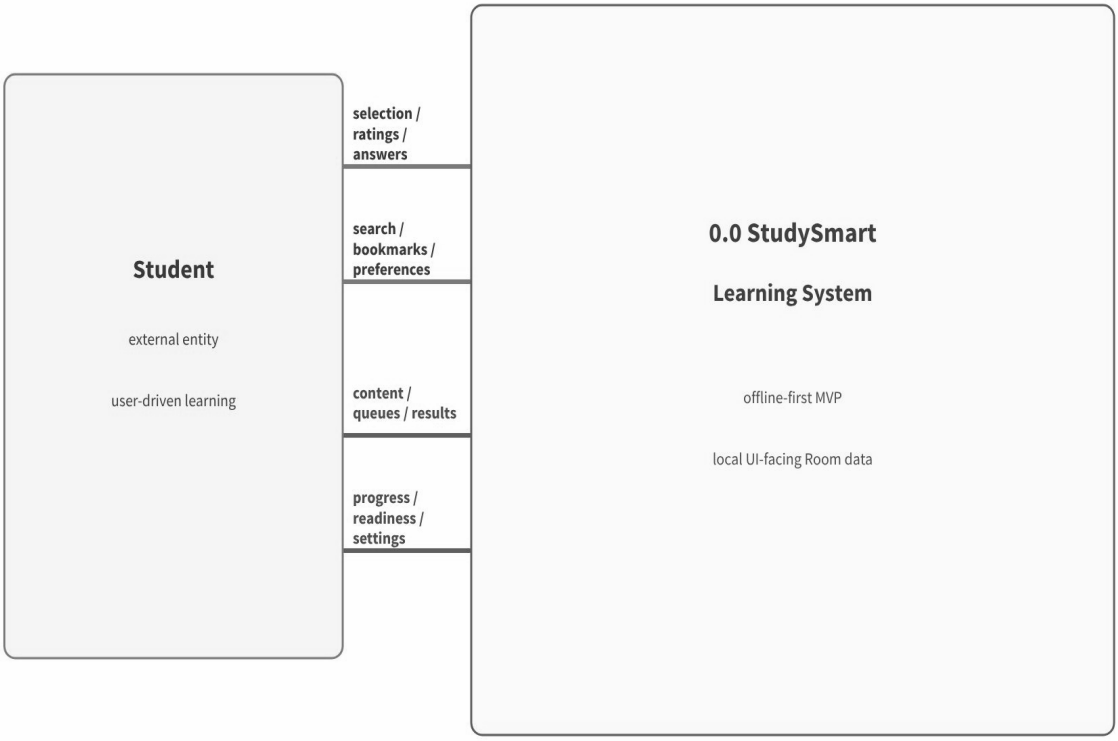


Figure 3.12: Context Diagram / Level 0 DFD

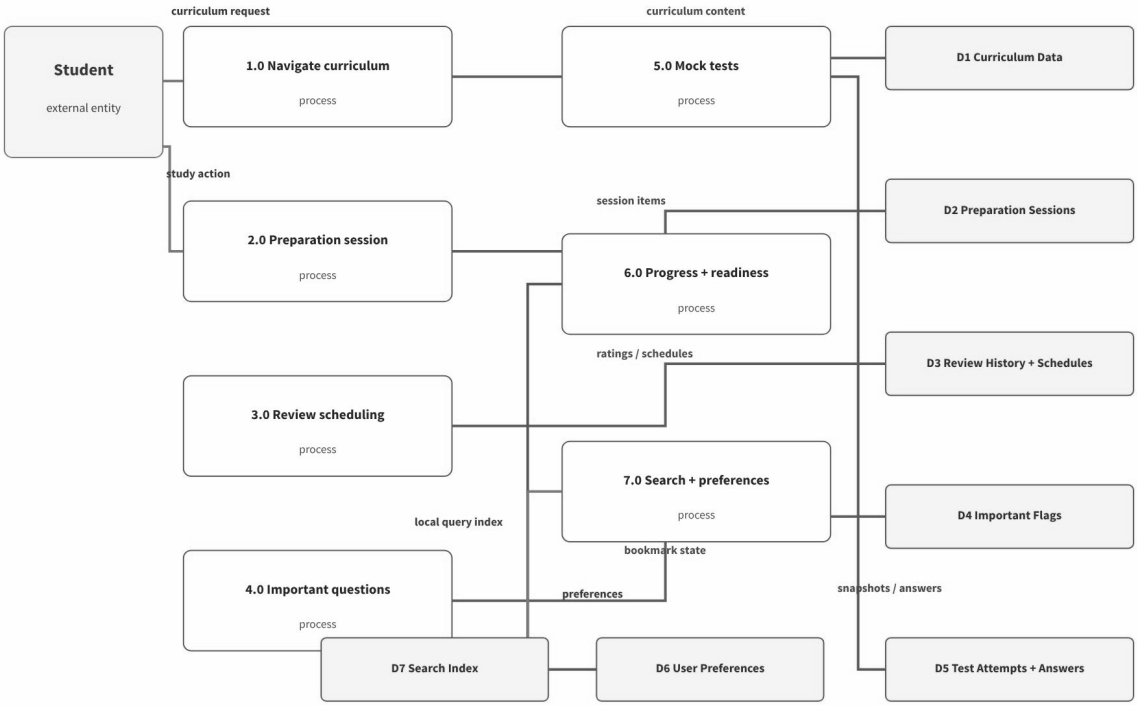


Figure 3.13: Level 1 Data Flow Diagram

3.2 System Design

Model Refinement and Architecture

StudySmart separates Compose UI, ViewModels with immutable StateFlow, domain services, repositories, and Room/DataStore, following Android's separation-of-responsibilities guidance [6]. Composables do not access Room directly; ViewModels do not contain database logic. Model refinements include stable IDs, optional topic ownership, transaction boundaries, and fixed session/test snapshots. The existing diagrams are referenced once rather than repeated as duplicate refined drawings.

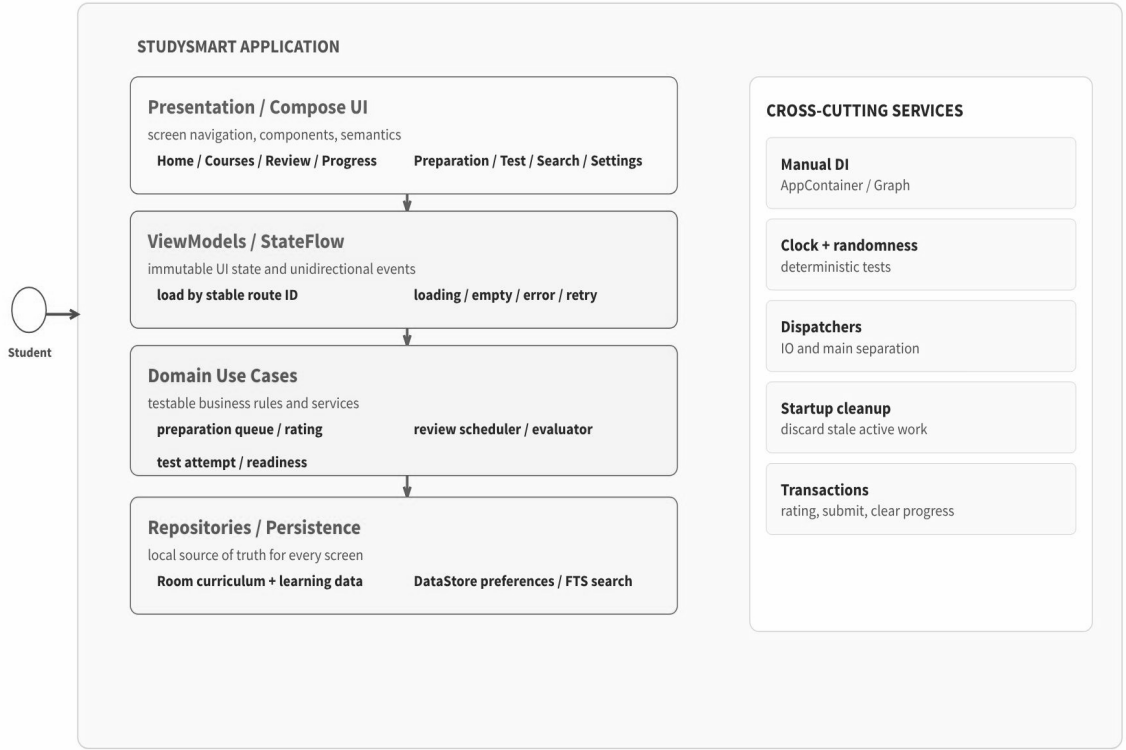


Figure 3.14: Layered Application Architecture

Component and Deployment Design

The architecture groups presentation, state handling, domain services, and local persistence, with AppContainer wiring the dependencies. The application is packaged as an Android APK and stores curriculum and learning data on the device; no backend server is part of Core Alpha. The supplied document contains no separate UML component or deployment diagram, so the retained architecture is not relabelled as either.

Database Design

Room holds curriculum and learning records; DataStore holds small preferences. The supplied ERD represents a broader design and includes items outside Core Alpha, such as user_question_flags and user_preferences. For the reported implementation, preferences are in DataStore, the seeded local user is an ownership record rather than an authentication account, and the implemented entity responsibilities are summarized below.

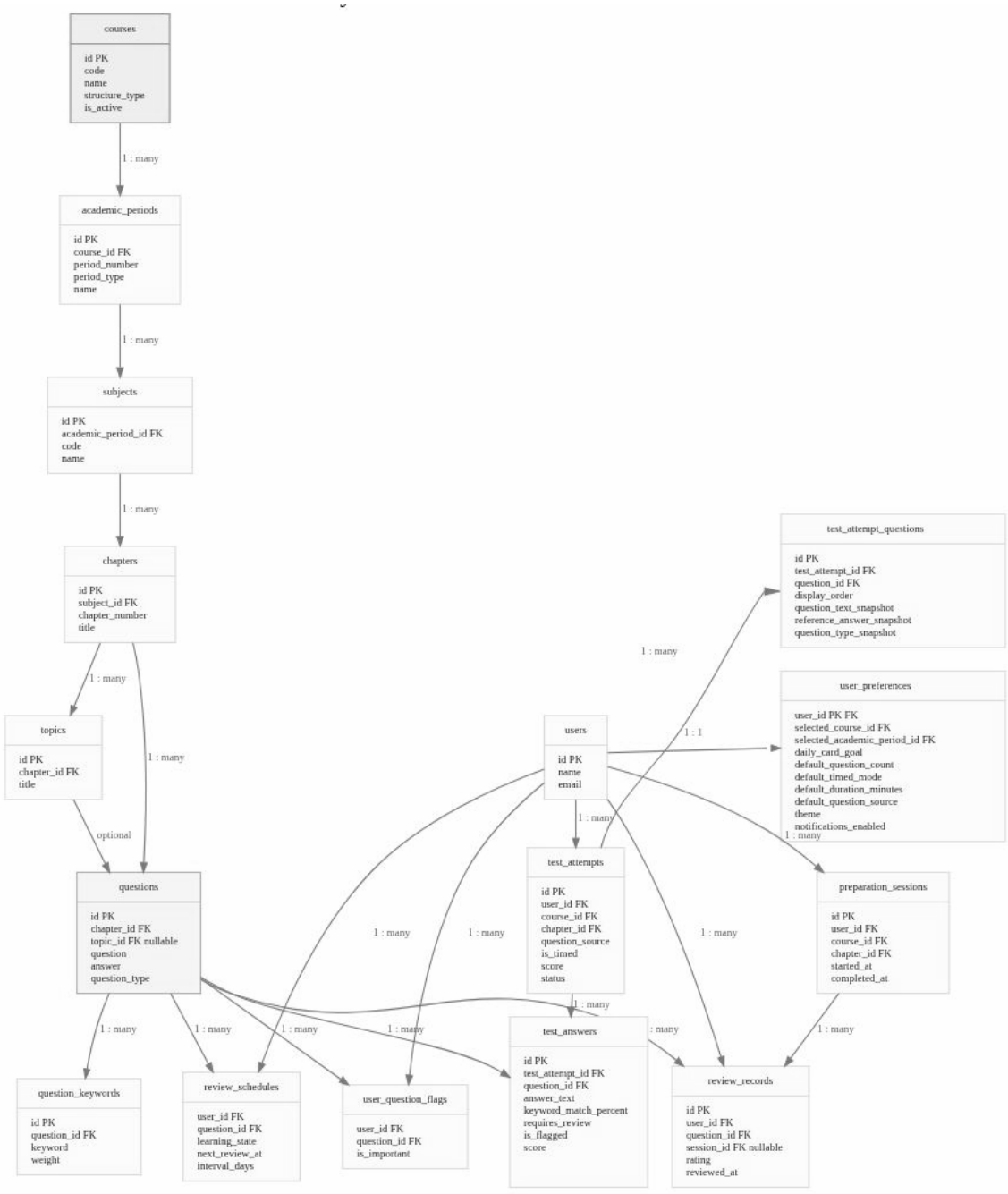


Figure 3.15: Supplied Database ERD (Broader Design)

Table 3.10: Core Alpha Data Entities

Entity	Responsibility / Relationship
users	Seeded local-user record owns learning data; no login identity.
courses	Academic programme and semester/year structure.
academic_periods	Stored periods linked to a course.
subjects	Subjects linked to an academic period.
chapters	Chapters linked to a subject.
topics	Optional subdivisions linked to a chapter.
questions	Chapter-owned written questions; optional topic; answer and maximum score.
question_keywords	Question-linked normalized keywords and positive weights.
preparation_sessions	Scope, origin, request ID, seed, lifecycle, and session counts.
preparation_session_items	Fixed question/answer snapshots and queue/reveal/rating state.
review_records	Immutable committed rating events.
review_schedules	One current schedule per local user/question.
test_attempts	Fixed scope/defaults, lifecycle, timing, totals, and evaluator version.
test_attempt_questions	Ordered frozen question, answer, keyword, and maximum-score snapshots.
test_answers	Draft text, flag/time data, keyword evidence, scores, and review decision.

Constraints reject invalid hierarchy links, empty required text, invalid weights/scores, and percentages outside 0–100. Only active questions with active ancestors are eligible. Rating writes history, schedule, queue state, and counts atomically; attempt creation and submission commit their related rows atomically. Startup cleanup removes transient active work without intentionally deleting committed history.

Interface Design

Navigation carries stable IDs through academic browsing, preparation, review, mock tests, and results. Data-driven screens account for loading, success, empty, missing/stale

IDs, errors/retry, pending writes, discard confirmation, and expired tests. Responsive Compose layouts support scrolling, insets, labels, and font scaling. The Penpot image below is a design artifact, distinct from the app screenshots in Appendix A.

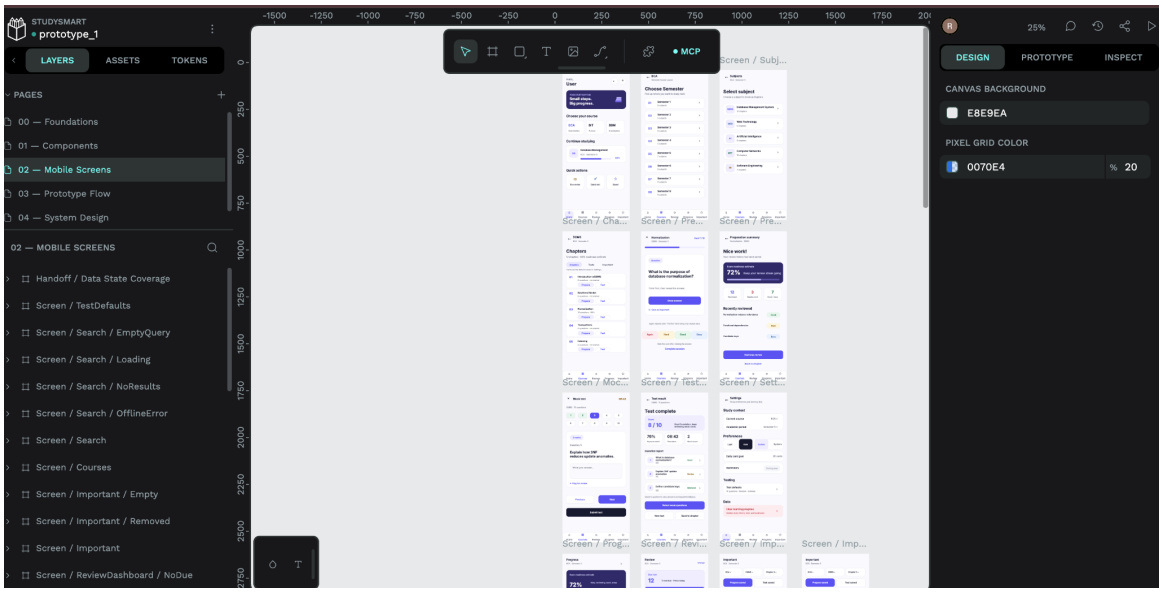


Figure 3.16: Penpot Interface Design Reference

The retained architecture and design images also show broader features such as search, bookmarks, and full progress. Their presence does not change the Core Alpha boundaries defined in Section 1.4. Privacy relies on the Android application sandbox and device protection; this is not an authentication or runtime security certification.

3.3 Algorithm Details

Preparation Selection and Queue

Filter eligible active questions, classify them as weak, due, unseen, or strong, order by that priority with a persisted seed for ties, and select at most 30 unique questions. Persist the fixed session before navigation. Again requeues after three pending presentations; only the first Hard rating requeues after six. Good and Easy do not requeue. Each committed rating creates an immutable review event. Repeated presentations do not increase the unique-question count.

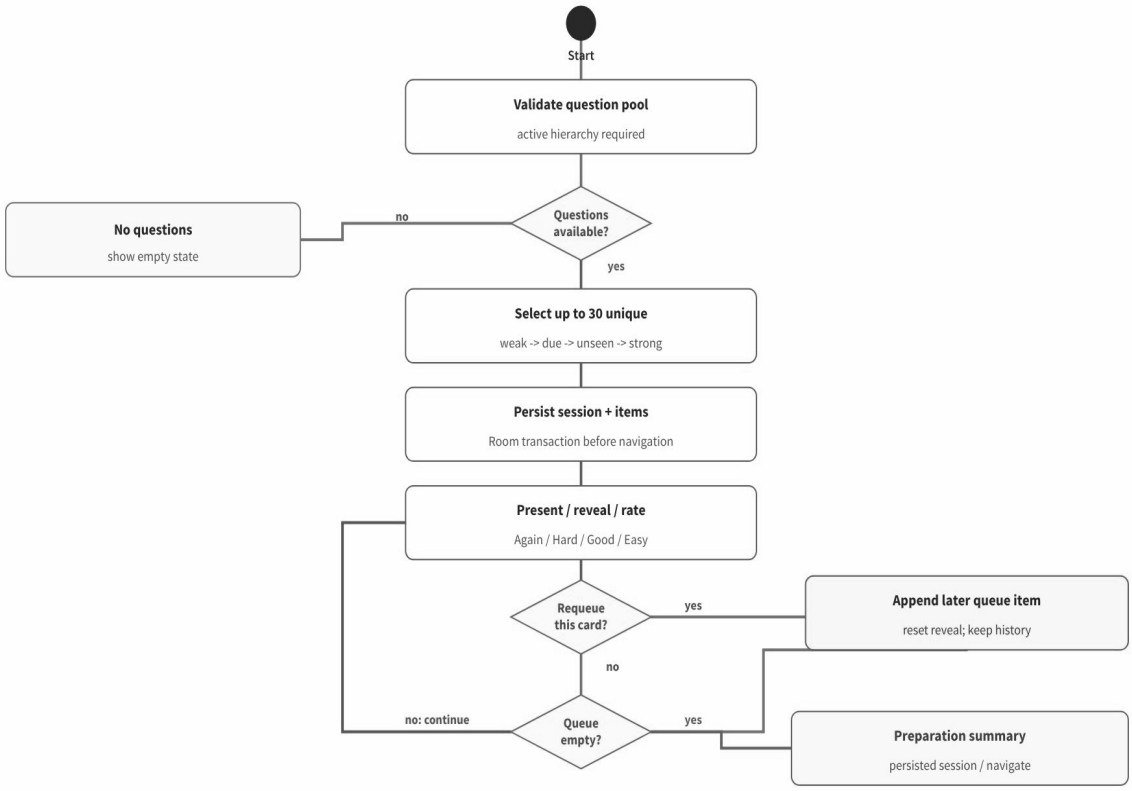


Figure 3.17: Preparation Selection and Queue Flowchart

Review Scheduling

Table 3.11: Deterministic Review Intervals

Rating	Interval
Again	10 minutes
Hard	1 day
Good	3 days
Easy	7 days

The scheduler uses an injected clock and stores the exact next-review instant. Due queries use that instant within the selected academic context.

Mock-Test Selection and Lifecycle

Defaults validate a question count of 1–30 and a timed duration of 1–180 minutes. Core Alpha enables all and random question sources. Eligible written questions are selected in stable order or using a persisted random seed. Empty pools create no attempt; a smaller

pool creates a smaller valid test. Attempt creation stores frozen questions and empty answer rows in one transaction.

The lifecycle is created → in_progress → submitting → submitted. Answer drafts are saved before navigation and confirmed submission. Expiry freezes the draft and offers Submit or Discard; it does not automatically submit. Submission is idempotent, and submitted attempts are read-only. This follows the recorded expiry test behaviour rather than ambiguous timeout wording in the earlier draft.

Keyword Matching and Scoring

Normalize the written response using Unicode NFKC, case folding, trim, and whitespace collapse. Match single keywords as whole words and phrases as contiguous normalized text; count each keyword once. Evaluation uses frozen positive-weight keywords.

keyword_match = matched_keyword_weight / total_keyword_weight × 100

question_score = roundHalfUp(max_score × keyword_match / 100, 2)

Blank responses with keywords receive zero match and score. Questions without keywords are not_evaluable: match is null, score is zero, and review is required. Matches below 50% require review. Attempt percentage is the bounded total score divided by total maximum score. Results describe keyword evidence, not semantic correctness.

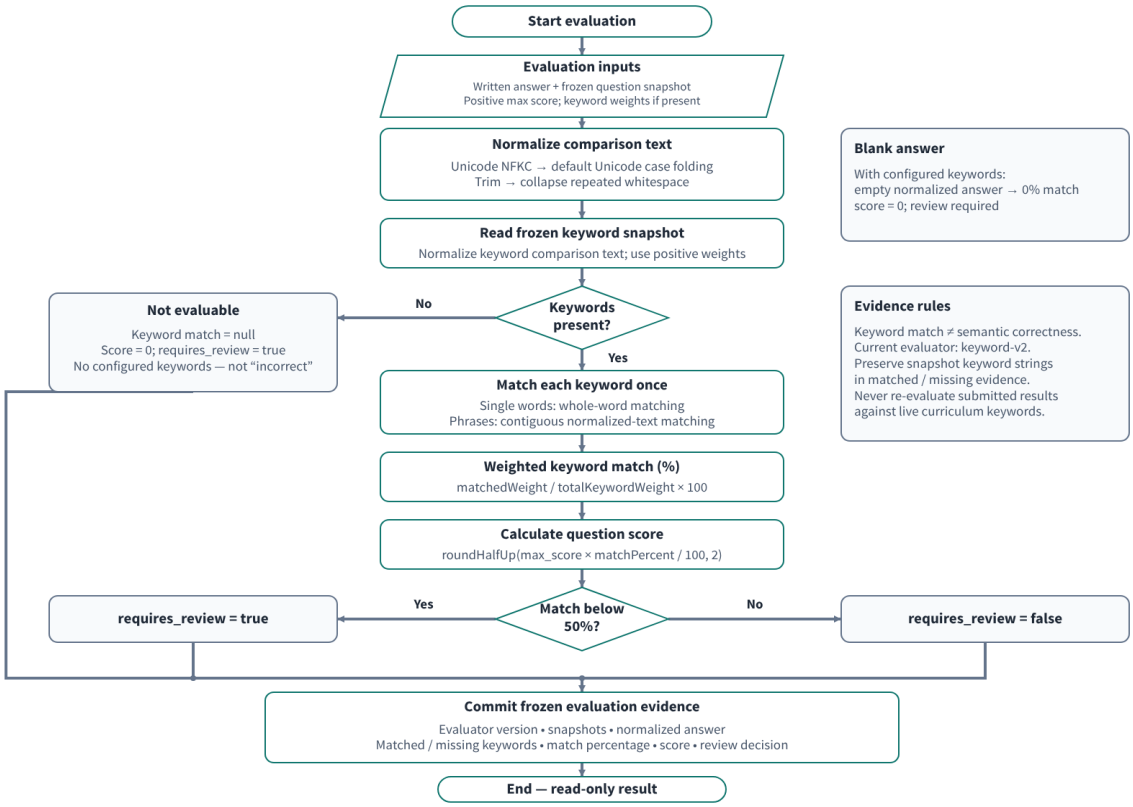


Figure 3.18: Keyword Evaluation Flowchart

Readiness Estimate

readiness = 0.40 × mastery + 0.35 × recent_test_score + 0.15 × coverage + 0.10 × review_freshness

The estimate is bounded to 0–100 and appears only after evaluated submitted-test data exists. It is not a scientifically validated prediction of examination performance.

Chapter 4: Implementation and Testing

4.1 Implementation

Tools Used

Table 4.1: Implementation Tools and Platforms

Purpose	Tool / Configuration
Language / IDE	Kotlin 2.3.21; Android Studio
UI	Jetpack Compose / Material 3; Compose BOM 2025.12.01 [7]
Navigation	Navigation Compose 2.9.6
Database	Room 2.8.5, SQLite; KSP 2.3.12 [8]
Preferences	Preferences DataStore 1.2.1 [9]
Asynchronous state	Coroutines, Flow, StateFlow, ViewModel [10]
Build	Gradle 8.13; AGP 8.13.2; Java/JDK 21
Android platform	Minimum API 26; compile/target API 35
Design / CASE artifacts	Penpot interface reference; supplied UML/DFD/ERD diagrams

Recorded versions are taken from the supplied report, not a fresh repository audit in this document workspace. The reported development machine was a MacBook Air M1 with 8 GB RAM. The recorded verification used a separate Debian/JDK environment; these descriptions are not merged into a claim about one workstation.

Implementation Details of Modules

Table 4.2: Module Implementation Responsibilities

Module / Boundary	Implementation Detail
Application foundation	AppContainer and StudySmartApplication.createContainer() wire local services; RoomStartupCleanupCoordinator removes unfinished work.
Academic browsing	Curriculum repositories and browse ViewModels load stored hierarchy; AcademicPeriodType maps Semester/Year labels.
Preparation	PreparationSelectionEngine fixes candidates; PreparationQueueEngine handles reveal/rating/requeue state with transactional persistence.
Review	ReviewScheduler calculates deterministic dates; due-review queries reuse the preparation engine.
Test creation/defaults	PreferencesRepository validates defaults; repositories create a fixed attempt and snapshots before navigation.
Mock-test interaction	ViewModels manage written drafts, navigation, flags, timing, expiry choices, submit/discard, and submitted-state guards.
Evaluation/results	KeywordAnswerEvaluator computes frozen evidence; result projections show scores, time, per-question matches, and review requirements.
Weak retest	A submitted result supplies active review-required questions for a new independent attempt.
Readiness	ReadinessCalculator combines mastery, recent score, coverage, and review freshness into a labelled estimate.

The application uses package com.studysmart.app and the :app Gradle module. StudySmartDatabase is Room schema version 8, named studysmart.db; DataStore preferences use user_preferences. Manual dependency wiring keeps UI, domain rules, repositories, and persistence separate. No backend or external scoring API is implemented in Core Alpha. Appendix B retains the supplied project-structure screenshot and source link.

4.2 Testing

Verification distinguishes pure/domain and fake-repository JVM checks from live Room, instrumentation, UI, and manual system checks. Passing isolated rules does not establish a complete installed-app workflow.

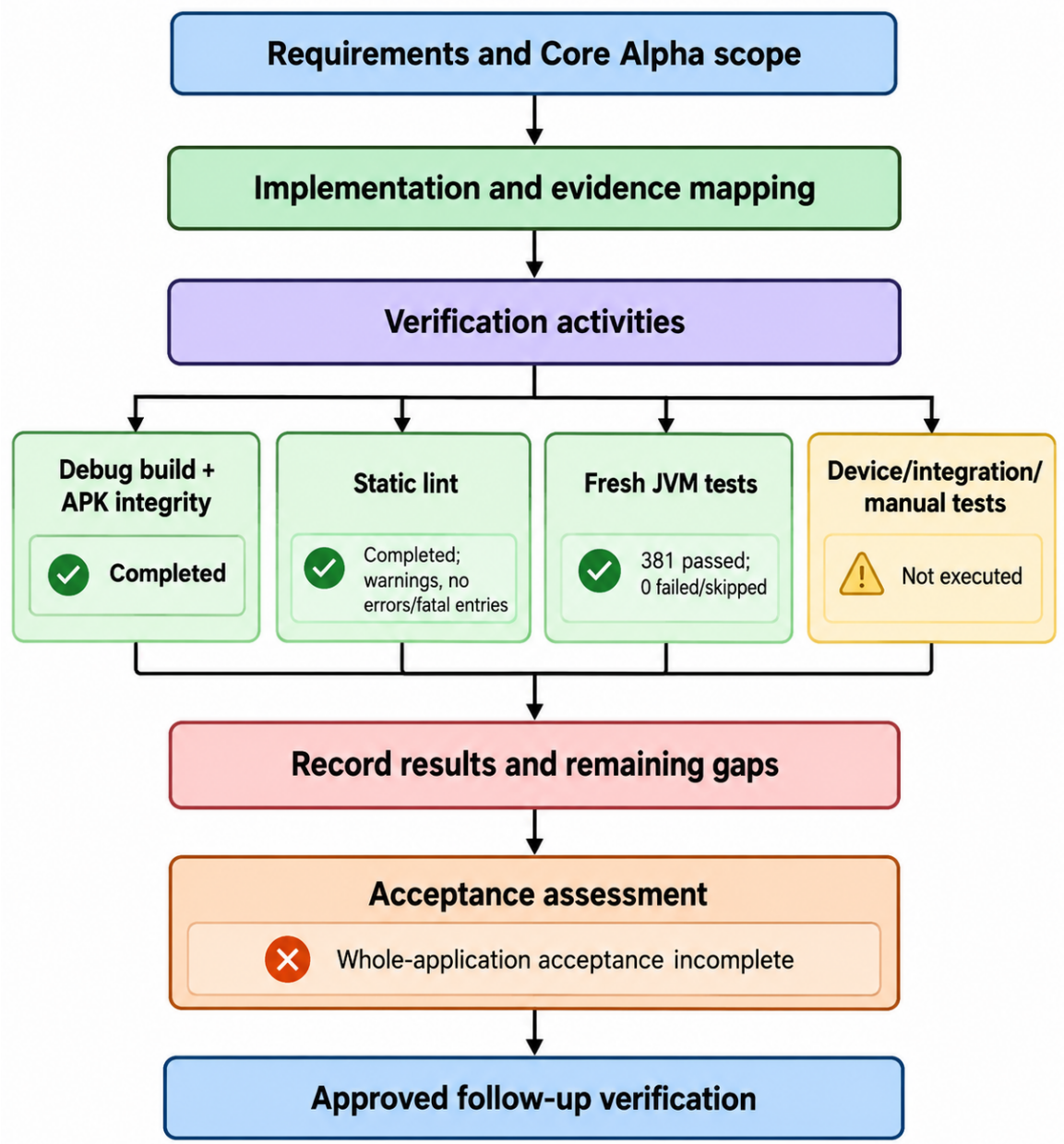


Figure 4.1: Testing and Verification Workflow

Test Cases for Unit Testing

The following representative input/expected-output descriptions follow the documented rules. Execution evidence is reported at suite level; individual test inputs and JUnit

records were not supplied with this DOCX. Suite totals below must not be summed as a complete test inventory.

Table 4.3: Representative Unit-Test Cases and Reported Evidence

Case / Area	**Representative Input → Expected Result**	**Reported Evidence**
UT-01: Period mapping	SEMESTER / YEAR → Choose Semester / Choose Year.	AcademicPeriodTypeTest: 2 cases passed.
UT-02: Selection	More than 30 eligible candidates → bounded unique selection in priority order.	PreparationSelectionEngine Test: 12 cases passed.
UT-03: Queue	Again / first Hard / Good / Easy → requeue after 3 / 6 / no / no pending presentations.	PreparationQueueEngineTe st: 13 cases passed.
UT-04: Scheduling	Fixed clock and rating → corresponding exact 10-minute/ 1-day/3-day/7-day due instant.	ReviewSchedulerTest: 6 cases passed.
UT-05: Evaluation	Written text and positive-weight keywords → normalized, once-counted weighted matches and bounded score.	KeywordAnswerEvaluatorT est: 9 cases passed.
UT-06: No keywords	Response with empty keyword set → null match, zero score, not_evaluable, review required.	Zero-keyword evaluator cases reported passed.
UT-07: Scoring	Frozen question scores/maxima → aggregate total and bounded percentage.	Isolated evaluator cases passed; live submission unverified.
UT-08: Readiness	Learning/test inputs → bounded 0–100 estimate using documented weights.	ReadinessCalculatorTest: 7 cases passed.

Case / Area	Representative Input → Expected Result	Reported Evidence
UT-09: Transformation	Curriculum rows → ordered contextual browse models.	CurriculumRepositoryTest.browseRowsBecomeOrderedContextualModels reported passed; live DAO query unverified.

Test Cases for System Testing

All cases in Table 4.4 were not executed during the final audit and require manual verification. Actual device outcomes are therefore not recorded. The consolidated matrix retains FT-01–FT-23 and four distinct boundary cases from the earlier TC matrix; overlapping cases are not counted as additional executed tests.

Table 4.4: Consolidated System-Test Cases — Manual Verification Required

ID / Scenario	Input / Action	Expected Result
FT-01: App startup	Launch debug app	Startup completes and Home appears
FT-02: Semester course	Select BCA and period	Semester labels and correct context
FT-03: Year course	Select yearly fixture and year	Year labels and correct context
FT-04: Subjects/chapters	Open subject then chapter	Correct child lists/details
FT-05: Session creation	Prepare a chapter	Persisted session loads fixed unique pool
FT-06: Reveal	Reveal answer	Answer visible, ratings enabled
FT-07: Again	Reveal, rate Again	Rating history is committed; Again requeues after three pending presentations.

ID / Scenario	Input / Action	Expected Result
FT-08: Hard	Rate Hard twice across repeats	First requeue after six; no second Hard requeue
FT-09: Good	Reveal, rate Good	No mandatory repeat
FT-10: Easy	Reveal, rate Easy	No mandatory repeat
FT-11: Due flow	Open due reviews	Only eligible due pool creates session
FT-12: Test defaults	Change count/source/timing	Confirmed preferences persist
FT-13: Attempt creation	Chapter Test	Persisted fixed questions loaded
FT-14: Written entry	Type answer, move away/ back	Draft restored
FT-15: Question navigation	Next/previous/jump	Correct question, saved draft
FT-16: Flagging	Flag then revisit	Flag retained in active attempt
FT-17: Timer	Create timed attempt, let expire	Expiry freezes the draft and offers Submit or Discard; no automatic submission.
FT-18: Submission	Confirm submit	One committed evaluated result
FT-19: Keyword evaluation	Weighted/blank answers	Transparent matching, scores and evidence
FT-20: Result screen	Open submitted result	Frozen answers and aggregates
FT-21: Weak retest	Retest weak from result	New attempt targets weak questions
FT-22: Restart	Restart after committed ratings/result	Committed learning and submitted results remain

ID / Scenario	Input / Action	Expected Result
FT-23: Active work restart	Restart with unfinished session/test	Active work removed, committed work unchanged
TC-007: Exceed unique limit	More than 30 eligible questions	Only 30 unique questions enter session
TC-009: No due questions	Empty due pool	NoDue state appears without false retry
TC-011: No eligible test questions	Empty pool	NoTestQuestions appears and no attempt is created
TC-015: Zero keywords	Question without keywords	Not-evaluable result and review requirement appear

Additional manual verification covers stale route IDs, invalid count/duration defaults, persistence failures and retries, duplicate submission, accessibility, backup/transfer exclusion, and runtime logs. The previously specified warm-start <1 s, cold-start <2.5 s, and local interaction <200 ms budgets remain unmeasured targets. No participant UAT dataset or feedback-driven acceptance result was established.

4.3 Result Analysis

Table 4.5: Recorded Verification Results

Check	Recorded Result / Scope
Debug build	:app:assembleDebug succeeded on 6 October 2026; app-debug.apk, version 0.1.0 (code 1).
Static lint	:app:lintDebug succeeded: 30 warnings, 0 errors, 0 fatal entries. Warnings are not necessarily distinct defects.
JVM unit tests	:app:testDebugUnitTest --rerun-tasks: 381 executed, 381 passed, 0 failed, 0 skipped.
APK curriculum integrity	:app:verifyDebugCurriculumApk passed inventory/hash checks.
Device/system acceptance	Instrumentation, live Room integration, and manual system checks were not completed in the final audit.

These reported results support buildability and selected domain/state behaviour, not production readiness, semantic grading accuracy, or complete persistence/UI acceptance. Earlier team-reported device verification had no established date, build revision, or workflow records and is not counted as final-audit acceptance evidence. Formal benchmarking and participant UAT remain unevidenced.

Chapter 5: Conclusion and Future Recommendations

5.1 Conclusion

StudySmart Core Alpha implements a focused offline Android learning workflow linking academic browsing, preparation, deterministic review, fixed written tests, and keyword-based result evidence. Its layered architecture separates UI, domain services, and local persistence. Recorded build, lint, curriculum-integrity checks, and 381 passing JVM cases support the scoped implementation. Live persistence, UI/system acceptance, participant UAT, and performance validation remain outstanding. The application is therefore an academic Core Alpha deliverable, not a fully verified production learning platform.

5.2 Future Recommendations

• Complete live Room, restart, navigation, accessibility, security/privacy, and performance verification, followed by recorded participant UAT.

• Expand bookmarks, filtered review, progress dashboards, search, settings, and completed-history presentation.

• Improve and validate curriculum coverage and source permissions; add managed content updates and further question types.

• Consider accounts, backend synchronization, backup, reminders, and teacher/admin content management only as later scope.

• Evaluate more capable answer assessment and readiness methods without presenting keyword match as semantic correctness.

References

1. H. L. Roediger III and J. D. Karpicke, “Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention,” Psychological Science, vol. 17, no. 3, pp. 249–255, 2006, doi: 10.1111/j.1467-9280.2006.01693.x.
2. N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer, “Distributed Practice in Verbal Recall Tasks: A Review and Quantitative Synthesis,” Psychological Bulletin, vol. 132, no. 3, pp. 354–380, 2006, doi: 10.1037/0033-2909.132.3.354.
3. H. Crompton and D. Burke, “The Use of Mobile Learning in Higher Education: A Systematic Review,” Computers & Education, vol. 123, pp. 53–64, 2018, doi: 10.1016/j.compedu.2018.04.007.
4. Anki, “Anki – Powerful, Intelligent Flashcards,” AnkiWeb. [Online]. Available: https://apps.ankiweb.net/
5. Quizlet, “Flashcards, Learning Tools, and Textbook Solutions,” Quizlet. [Online]. Available: https://quizlet.com/en/features/flashcards
6. Android Developers, “Guide to App Architecture,” Android Developers, Google. [Online]. Available: https://developer.android.com/topic/architecture
7. Android Developers, “Get Started with Jetpack Compose,” Android Developers, Google. [Online]. Available: https://developer.android.com/develop/ui/compose/documentation
8. Android Developers, “Save Data in a Local Database Using Room,” Android Developers, Google. [Online]. Available: https://developer.android.com/training/data-storage/room
9. Android Developers, “DataStore,” Android Developers, Google. [Online]. Available: https://developer.android.com/topic/libraries/architecture/datastore
10. JetBrains, “Coroutines Guide,” Kotlin Documentation. [Online]. Available: https://kotlinlang.org/docs/coroutines-guide.html

Appendices

Appendix A: Application Screenshots

These are supplied screenshots of the running application, as confirmed by the project team. Screens showing broader functionality do not change the reported Core Alpha scope or establish that manual system tests passed.

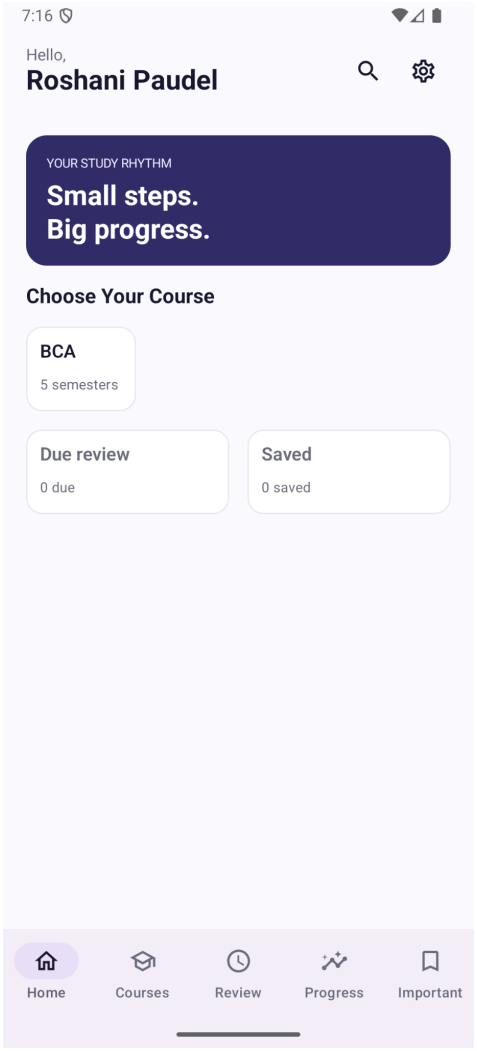


Figure A.1: Application Home / Navigation

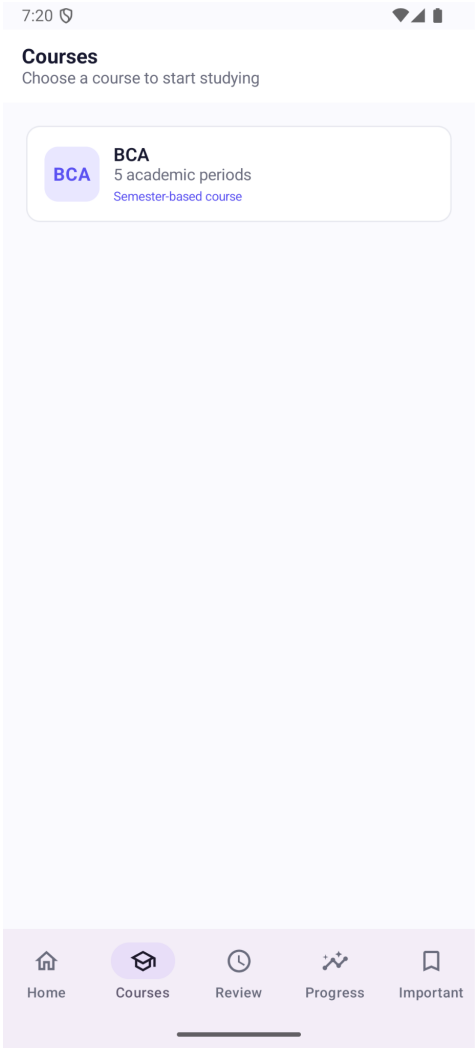


Figure A.2: Courses Screen

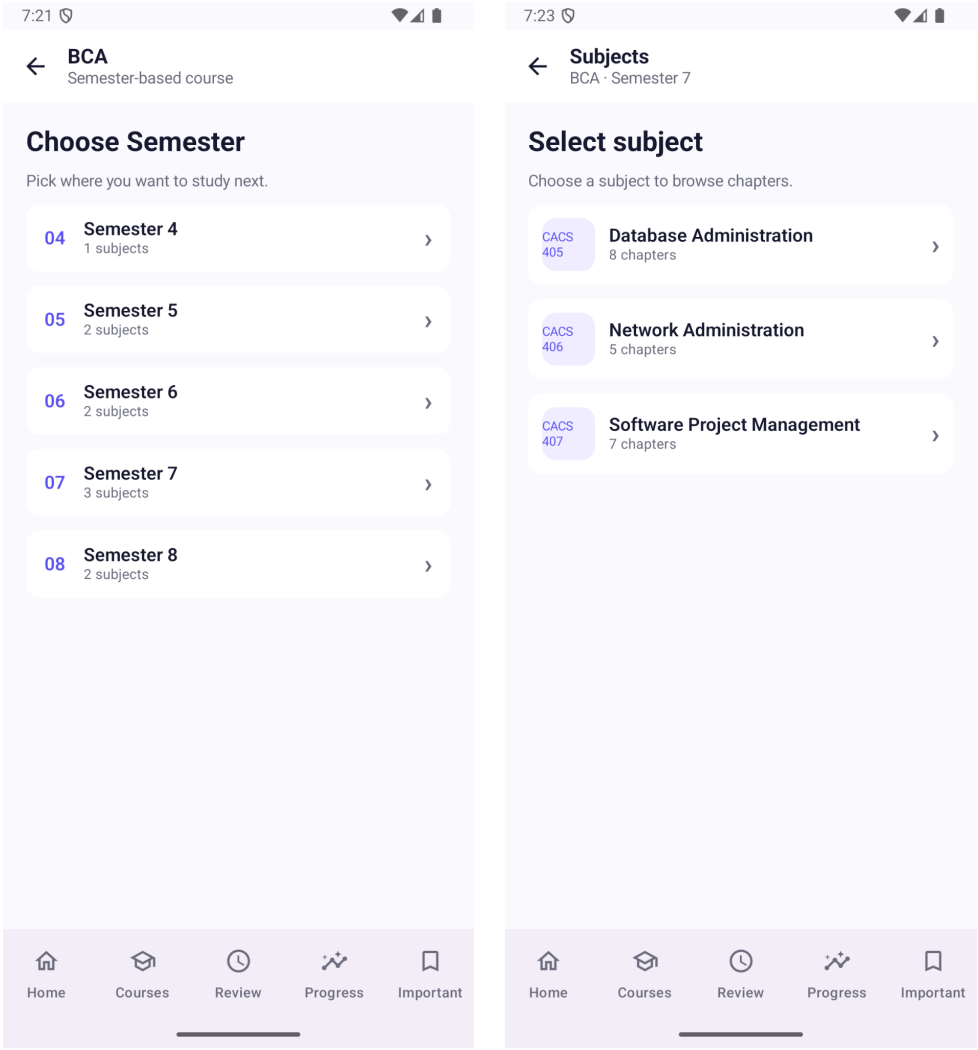


Figure A.3: Academic-Period Screen

Figure A.4: Subjects and Chapters Screen

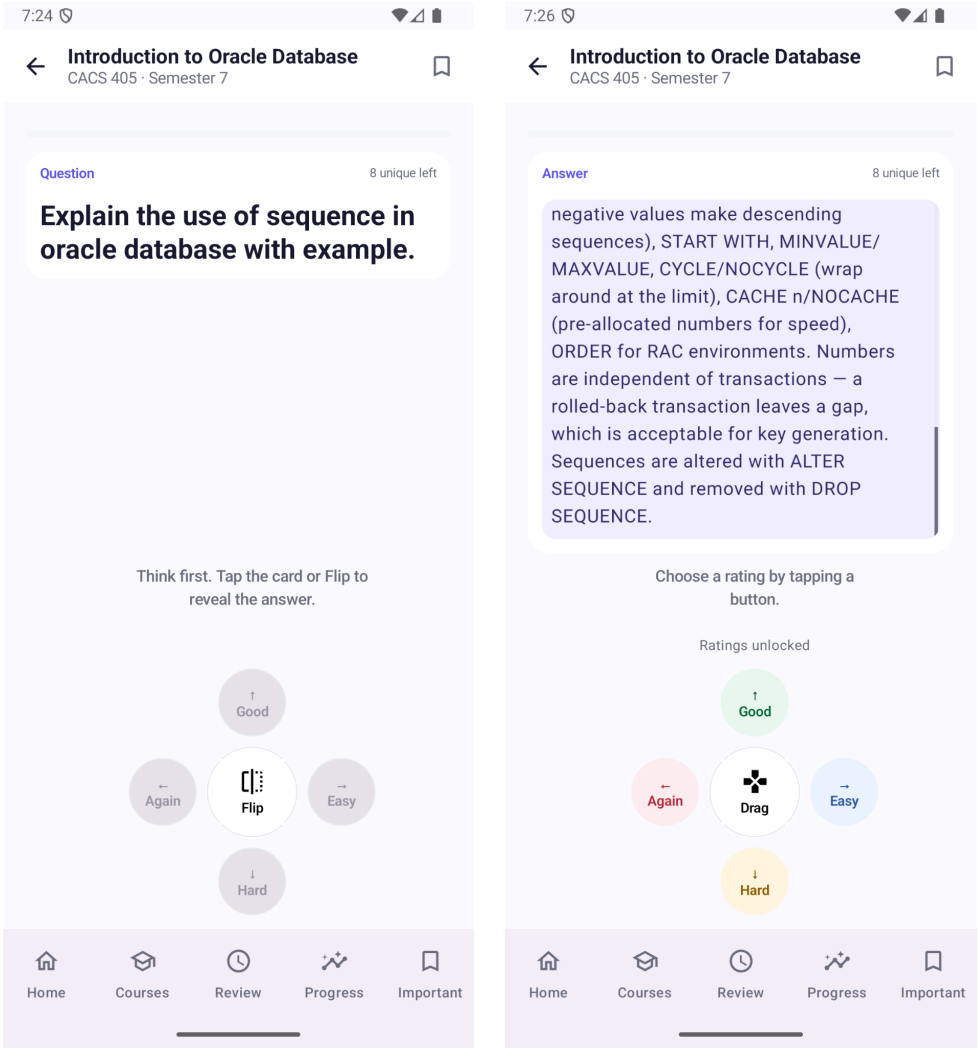


Figure A.5: Preparation Screens

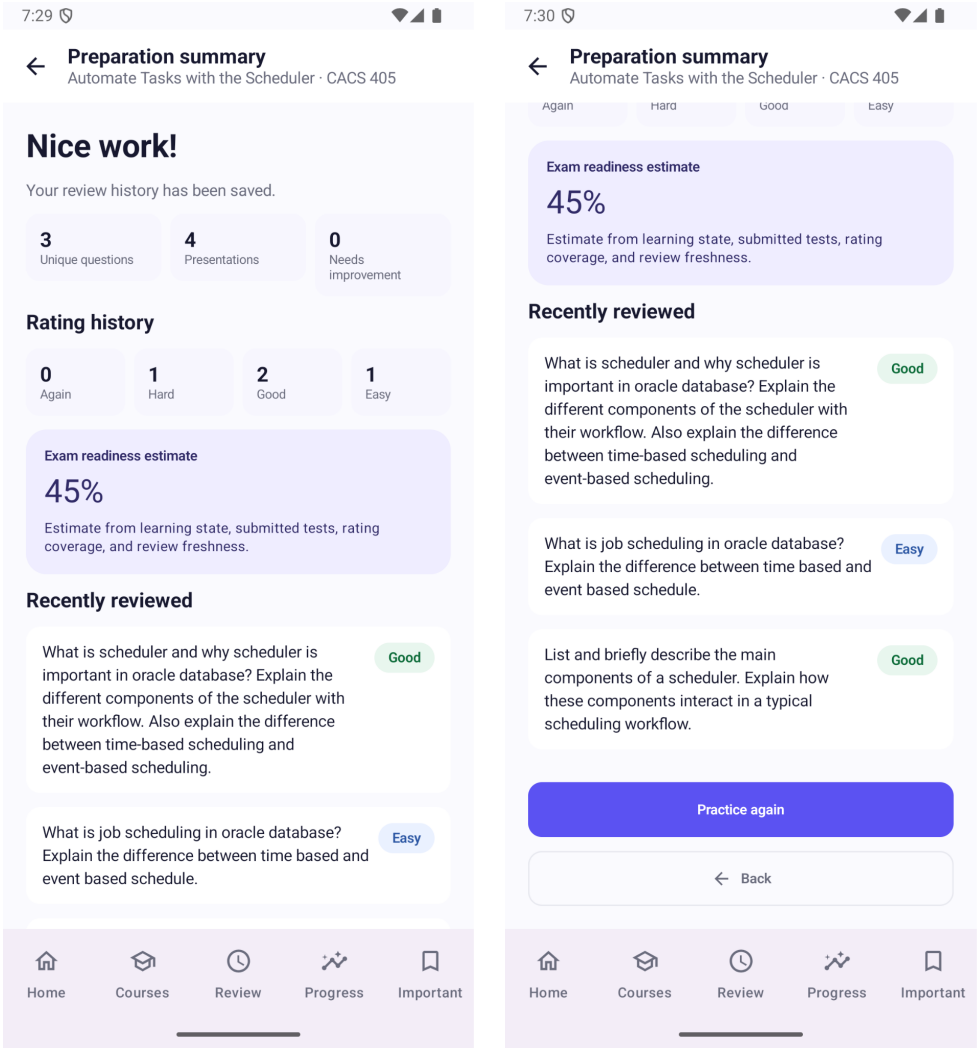


Figure A.6: Preparation Summary Screens

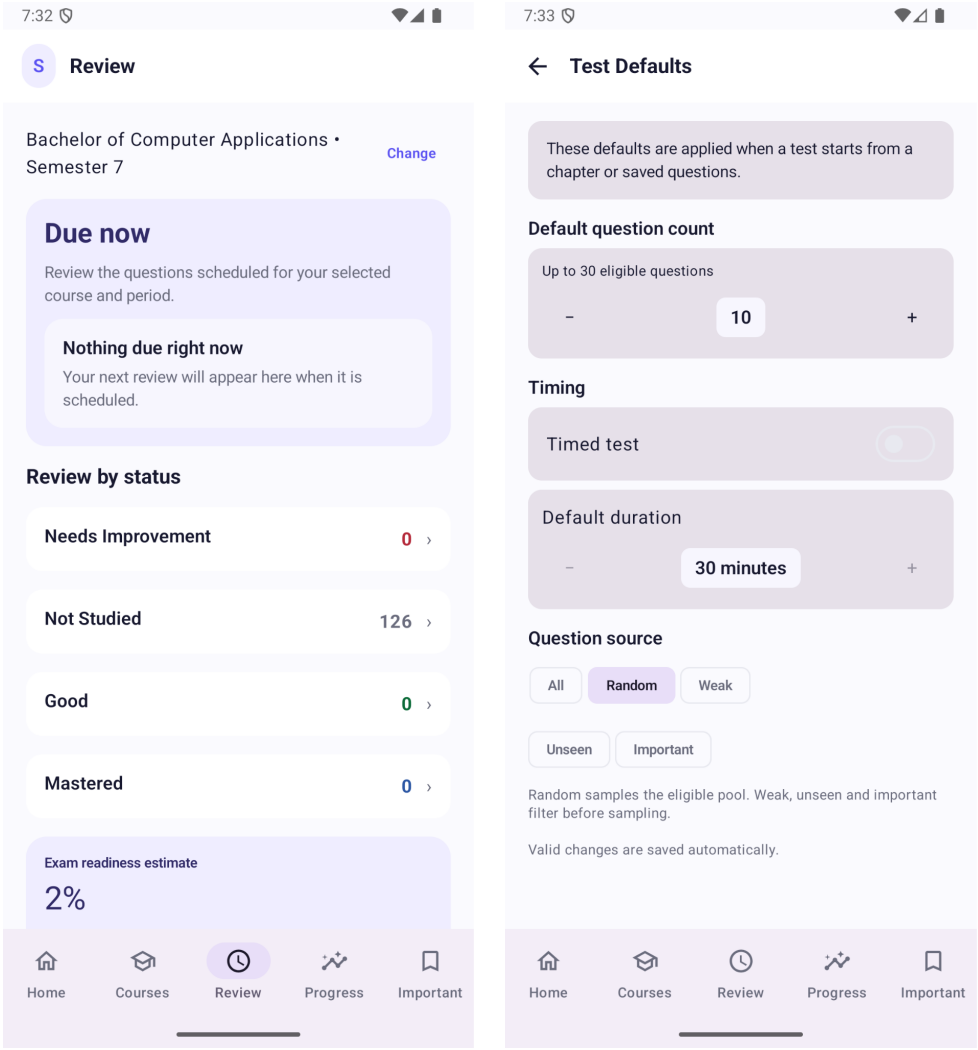


Figure A.7: Review Screen

Figure A.8: Test Defaults Screen

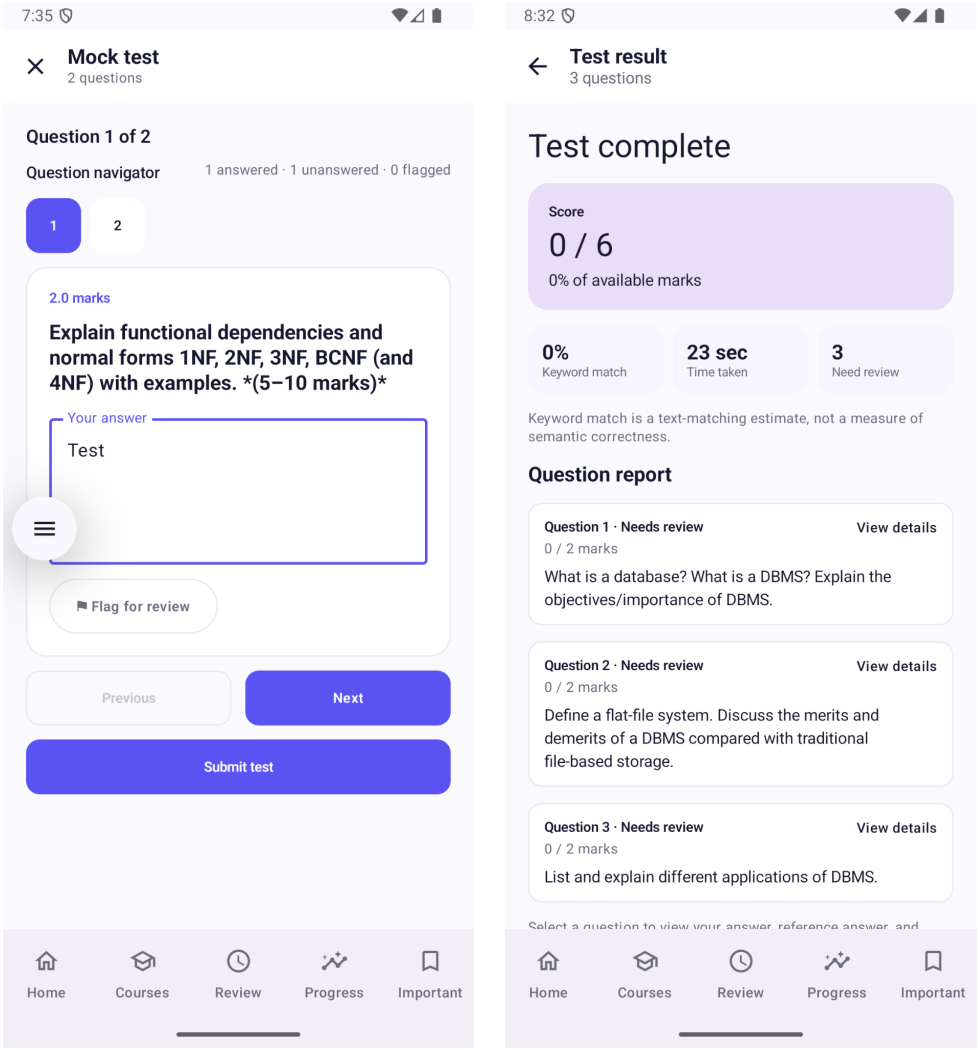
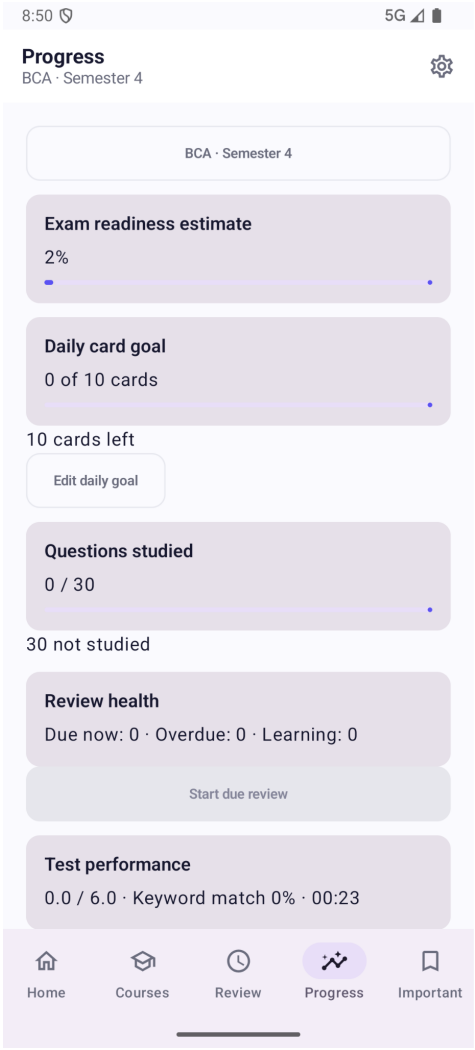


Figure A.9: Mock-Test Screen

Figure A.10: Test-Result Screen



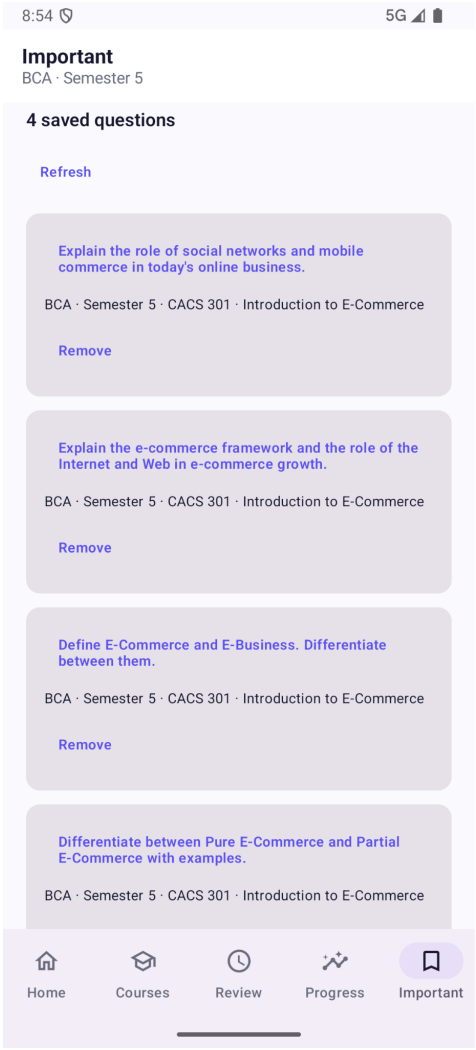
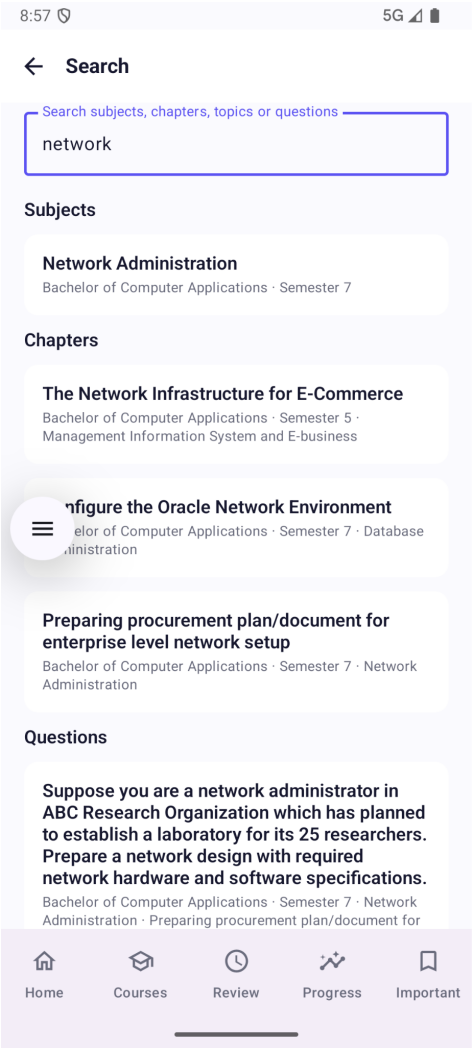


Figure A.11: Progress Screen (Broader Functionality)

Figure A.12: Important Questions Screen (Broader Functionality)



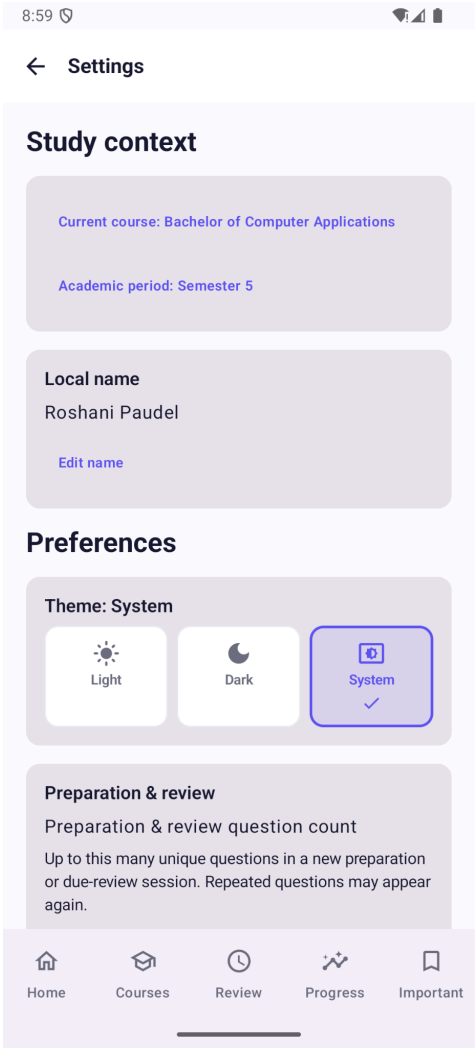


Figure A.13: Search Screen (Broader Functionality)

Figure A.14: Settings Screen (Broader Functionality)

Appendix B: Source Code

Project source: https://github.com/roshanipdl/studysmart/

The supplied IDE screenshot shows part of the project structure. Representative implementation boundaries are AppContainer, StudySmartDatabase, PreferencesRepository, PreparationSelectionEngine, PreparationQueueEngine, ReviewScheduler, KeywordAnswerEvaluator, and ReadinessCalculator. The screenshot includes broader modules and is not a complete inventory of the audited Core Alpha revision.

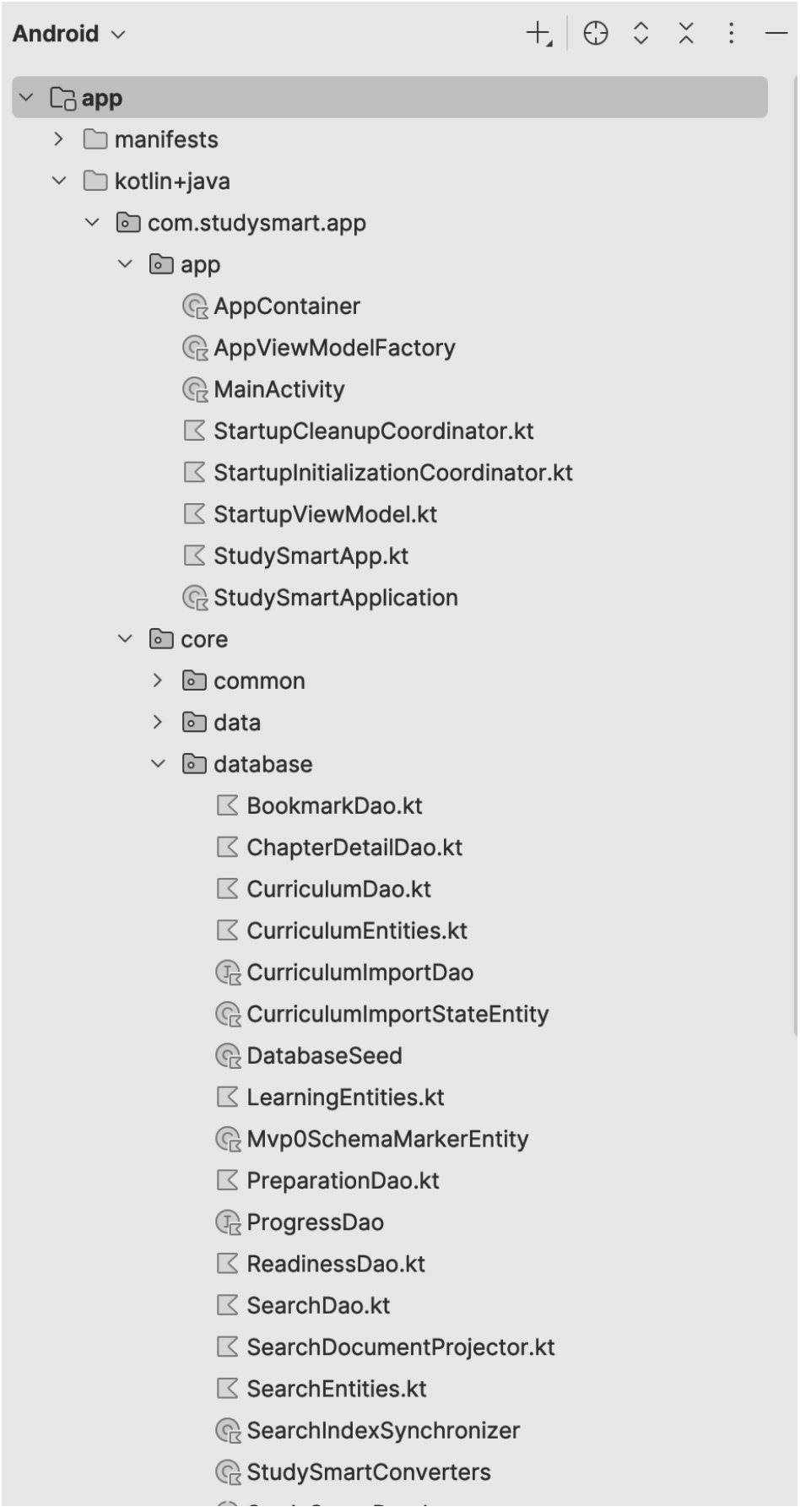


Figure B.1: Supplied Android Studio Project Structure